

# Uncertainty-Type-Driven Subtask Routing: Separating Epistemic from Aleatoric Signals in LLM Pipeline Orchestration

Master's Thesis  
of

Hymavathy Vanam  
5003852

Master of Science in Artificial Intelligence

This report is submitted to  
Chair of Wireless Systems  
Brandenburg University of Technology

Primary Supervisor: Prof. Dr. rer. nat. Peter Langendörfer   
Secondary Supervisor: Dr. rer. nat. Svetlana Meissner

Cottbus, 23.09.2026

# Contents

|                                                                               |             |
|-------------------------------------------------------------------------------|-------------|
| <b>Acknowledgments</b>                                                        | <b>iv</b>   |
| <b>Author’s Declaration</b>                                                   | <b>v</b>    |
| <b>Abstract</b>                                                               | <b>vi</b>   |
| <b>List of Figures</b>                                                        | <b>viii</b> |
| <b>List of Tables</b>                                                         | <b>ix</b>   |
| <b>List of Abbreviations</b>                                                  | <b>x</b>    |
| <b>1 Introduction</b>                                                         | <b>1</b>    |
| 1.1 Motivation . . . . .                                                      | 1           |
| 1.2 Problem Statement . . . . .                                               | 2           |
| 1.3 Proposed Approach: UTSR in Brief . . . . .                                | 3           |
| 1.4 Research Questions and Hypotheses . . . . .                               | 3           |
| 1.5 Contributions . . . . .                                                   | 4           |
| 1.6 Scope and Delimitations . . . . .                                         | 5           |
| 1.7 Thesis Outline . . . . .                                                  | 5           |
| <b>2 Background</b>                                                           | <b>6</b>    |
| 2.1 Two Kinds of Uncertainty . . . . .                                        | 6           |
| 2.1.1 The classical decomposition . . . . .                                   | 6           |
| 2.1.2 Why the decomposition matters for decision-making . . . . .             | 7           |
| 2.1.3 Estimating the components in classical deep learning . . . . .          | 7           |
| 2.2 Uncertainty Quantification for Large Language Models . . . . .            | 7           |
| 2.2.1 The calibration problem . . . . .                                       | 7           |
| 2.2.2 Sampling-based methods: self-consistency and semantic entropy . . . . . | 8           |
| 2.2.3 Verbalised and introspective confidence . . . . .                       | 8           |
| 2.2.4 Multi-model and multi-agent estimators . . . . .                        | 8           |
| 2.2.5 Abstention and selective prediction . . . . .                           | 9           |
| 2.3 Natural Language Inference and Semantic Clustering . . . . .              | 9           |
| 2.3.1 The Natural language Inference task . . . . .                           | 10          |
| 2.3.2 Semantic equivalence clustering of sampled answers . . . . .            | 10          |
| 2.3.3 From clusters to geometry . . . . .                                     | 10          |
| 2.4 LLM Agents, Tool Use, and Reflection . . . . .                            | 10          |
| 2.4.1 Tool-augmented generation . . . . .                                     | 11          |
| 2.4.2 Reflection and self-refinement . . . . .                                | 11          |
| 2.4.3 Multi-agent pipelines and orchestration . . . . .                       | 11          |
| 2.4.4 Error propagation in long-horizon pipelines . . . . .                   | 11          |
| 2.5 Human-in-the-Loop Escalation and Learning to Defer . . . . .              | 12          |
| 2.5.1 Learning to defer . . . . .                                             | 12          |
| 2.5.2 The cost of human oversight in agentic systems . . . . .                | 12          |

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| 2.5.3    | What a good escalation looks like . . . . .                                      | 12        |
| 2.6      | Benchmark Foundations: Multi-hop and Ambiguous QA . . . . .                      | 13        |
| 2.6.1    | HotpotQA: epistemic uncertainty by construction . . . . .                        | 13        |
| 2.6.2    | AmbigQA / AmbigNQ: aleatoric uncertainty by construction . . . . .               | 13        |
| 2.6.3    | Why the pairing is clean . . . . .                                               | 13        |
| <b>3</b> | <b>Related Work</b>                                                              | <b>14</b> |
| 3.1      | Scalar Uncertainty Quantification and Abstention . . . . .                       | 14        |
| 3.2      | Ambiguity in Question Answering . . . . .                                        | 14        |
| 3.3      | Uncertainty-Aware Agents and Agentic UQ . . . . .                                | 15        |
| 3.4      | Orchestration, Routing, and Human Escalation in Pipelines . . . . .              | 15        |
| 3.5      | Critiques of the Epistemic/Aleatoric Taxonomy for LLMs . . . . .                 | 16        |
| 3.6      | Gap Analysis and Positioning . . . . .                                           | 16        |
| <b>4</b> | <b>Methodology: The UTSR System Design and Experimental Setup</b>                | <b>18</b> |
| 4.1      | Research Questions . . . . .                                                     | 18        |
| 4.2      | System Design . . . . .                                                          | 18        |
| 4.2.1    | Design rationale . . . . .                                                       | 19        |
| 4.2.2    | The uncertainty-type classifier . . . . .                                        | 20        |
| 4.2.3    | Routing, specialist agents, and loop supervision . . . . .                       | 20        |
| 4.3      | Experimental Setup . . . . .                                                     | 21        |
| 4.3.1    | Evaluation models . . . . .                                                      | 22        |
| 4.3.2    | Benchmark construction . . . . .                                                 | 22        |
| 4.3.3    | The dataset filtering protocol . . . . .                                         | 23        |
| 4.3.4    | The length-confound audit . . . . .                                              | 23        |
| 4.3.5    | Baseline systems . . . . .                                                       | 24        |
| 4.3.6    | Metrics . . . . .                                                                | 24        |
| 4.3.7    | Configuration and protocol . . . . .                                             | 25        |
| <b>5</b> | <b>Experimental Evaluation and Results</b>                                       | <b>27</b> |
| 5.1      | Headline Comparison . . . . .                                                    | 27        |
| 5.2      | Classification Quality by Class . . . . .                                        | 28        |
| 5.3      | Failure-Mode Results . . . . .                                                   | 29        |
| 5.4      | Routing Behaviour . . . . .                                                      | 31        |
| 5.5      | Cross-Model Analysis: the Signal Depends on the Base Model’s Sampling Entropy    | 32        |
| 5.6      | Effect of the Dataset Filter . . . . .                                           | 34        |
| 5.7      | Length-Confound Audit . . . . .                                                  | 34        |
| 5.8      | Efficiency . . . . .                                                             | 35        |
| <b>6</b> | <b>Discussion</b>                                                                | <b>38</b> |
| 6.1      | The hypotheses revisited . . . . .                                               | 38        |
| 6.2      | Why the baselines win individual classes - and why those are the wrong metrics . | 38        |
| 6.3      | The delta test: sound logic, distribution-dependent input . . . . .              | 39        |
| 6.4      | False uncertainty on the certain controls: a shared clusterer failure . . . . .  | 39        |
| 6.5      | Label-level versus action-level performance . . . . .                            | 40        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 6.6      | Threats to validity revisited . . . . .        | 40        |
| <b>7</b> | <b>Conclusion and Future Work</b>              | <b>41</b> |
| 7.1      | Summary of the Thesis . . . . .                | 41        |
| 7.2      | Answers to the Research Questions . . . . .    | 42        |
| 7.3      | Contributions . . . . .                        | 42        |
| 7.4      | Limitations . . . . .                          | 43        |
| 7.5      | Future Work . . . . .                          | 44        |
| 7.6      | Closing Remarks . . . . .                      | 45        |
|          | <b>Appendix A - Core Implementation</b>        | <b>46</b> |
| A.1      | Hyperparameters and Thresholds . . . . .       | 46        |
| A.2      | Semantic Clustering . . . . .                  | 48        |
| A.3      | Uncertainty Classifier . . . . .               | 52        |
| A.4      | Uncertainty Signal and Routing Types . . . . . | 58        |
| A.5      | Router . . . . .                               | 59        |
| A.6      | Specialist Agents . . . . .                    | 65        |
|          | <b>Bibliography</b>                            | <b>70</b> |

# Acknowledgments

I would like to express my sincere gratitude to my supervisors, Prof. Dr. Peter Langendörfer and Dr. Svetlana Meissner, for their continuous guidance, valuable feedback, and academic support throughout the course of this Master's thesis. Their expertise, encouragement, and constructive criticism were instrumental in shaping the direction of this work and ensuring its scientific rigor.

I am also grateful to the Chair of Wireless Systems at Brandenburg University of Technology Cottbus-Senftenberg for providing the academic environment and resources necessary to carry out this research. The knowledge gained during my studies and research at the chair significantly contributed to the successful completion of this thesis.

Furthermore, I would like to thank my friends for the insightful discussions, technical suggestions, and moral support during the development of this work. Their perspectives and encouragement helped me overcome challenges encountered throughout the research process.

Finally, I would like to express my deepest appreciation to my family for their unwavering support, patience, and motivation throughout my academic journey. Their constant encouragement made it possible for me to pursue and complete this work successfully.

# Author's Declaration

*This declaration applies to term papers, seminar papers, essays, including Bachelor's and Master's theses*

*(cf. § 12 (9) and § 20 (4) RahmenO-BA/MA)*

I, Hymavathy Vanam, hereby declare that the thesis entitled Uncertainty-Type-Driven Subtask Routing: Separating Epistemic from Aleatoric Signals in LLM Pipeline Orchestration is my own original work, except where specific reference is made to the contributions of others. A true and complete copy of the thesis, including any necessary revisions, has been submitted to and accepted by my examiners. Proper acknowledgment has been given throughout the thesis wherever the work of others has been referenced. This thesis has not been submitted in the same or similar form, not even partially, within the scope of a different examination. Thus far, it has not yet been published. I agree that the work may be checked for plagiarism using a plagiarism detection service.

I further declare that I made use of artificial intelligence-based tools (e.g., AI language models such as Gemini and ChatGPT) to support the writing process of this thesis. The use of these tools was limited to assistance such as language refinement, restructuring of text, and improvement of clarity and readability. The scientific content, analysis, interpretation of results, and conclusions are entirely my own responsibility. I am aware that violating these principles may lead to the rejection of the thesis.

I furthermore declare that I comply with the “Statute for Safeguarding Good Scientific Practice and for Dealing with Cases of Suspected Scientific Misconduct at Brandenburg University of Technology Cottbus–Senftenberg dated 20 July 2023” ([https://opus4.kobv.de/opus4-btu/frontdoor/deliver/index/docId/6774/file/31\\_Satzung.pdf](https://opus4.kobv.de/opus4-btu/frontdoor/deliver/index/docId/6774/file/31_Satzung.pdf)).

Cottbus, 23.09.2026

---

Hymavathy Vanam

# Abstract

Large language model (LLM)-based multi-agent systems are increasingly deployed in structured workflow settings - compliance review, legal analysis, research pipelines, and customer escalation systems - where a task is decomposed across multiple specialised agents arranged in a defined processing order. A fundamental unsolved problem in these pipelines is what to do when an agent is uncertain about its output. Existing approaches route uncertain cases to the next agent in a fixed sequence or escalate them to a human reviewer, treating all uncertainty as a single undifferentiated signal that requires the same intervention regardless of its source. This conflation produces two systematic failure modes: wasted escalation, in which epistemically uncertain subtasks that could have been resolved by retrieval are sent to expensive human reviewers, and unresolvable reflection, in which aleatorically uncertain subtasks are trapped in reflection loops that consume tool calls and compute without ever converging, because the underlying question is genuinely ambiguous.

This thesis introduces Uncertainty-Type-Driven Subtask Routing (UTSR), a training-free runtime mechanism that classifies agent uncertainty as either epistemic (reducible through additional information) or aleatoric (irreducible ambiguity inherent to the task) and routes each subtask to a type-appropriate specialist agent. UTSR analyses the structural pattern of an agent’s output distribution across  $N$  sampled responses, clusters the samples semantically using a natural language inference (NLI) model, and classifies the uncertainty type from the resulting cluster geometry: outputs that scatter across many semantically distinct clusters indicate epistemic uncertainty, whereas outputs that polarise into a small number of semantically opposed camps indicate aleatoric uncertainty. Epistemic cases are routed to a retrieval agent with a context-injection test that empirically verifies whether uncertainty is in fact reducible; aleatoric cases are routed directly to a clarification agent that enumerates the valid interpretations for structured human review and issues zero retrieval calls; and a two-attempt retrieval budget with a behavioural re-routing rule terminates loops that fail to converge.

The approach is evaluated on a balanced benchmark constructed from HotpotQA (multi-hop questions whose uncertainty is epistemic by construction) and AmbigNQ (under-specified questions whose uncertainty is aleatoric by construction) against three baselines: a no-escalation agent, a fixed confidence-threshold router, and an agentic uncertainty-quantification reflection loop. A dual failure-mode evaluation framework separately measures wasted escalations on epistemic cases and wasted tool calls on aleatoric cases, alongside per-class classification F1 and end-to-end latency.

Across all three models UTSR attains the highest macro F1 (0.376-0.432 versus at most 0.259 for the best baseline), is the only system with non-zero aleatoric detection, strictly dominates the fixed-threshold router on both wasted and missed escalations, and eliminates the 48-67 futile tool calls per model that the reflection baseline expends on ambiguous items. A cross-model analysis shows that the geometric signal is a function of the base model’s sampling entropy: aleatoric F1 falls ( $0.612 \rightarrow 0.278$ ) and epistemic F1 rises ( $0.167 \rightarrow 0.475$ ) with model capability, so the mechanism transfers across models but a single fixed configuration does not.

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | The UTSR pipeline. A question is sampled N times without context and the samples are clustered under semantic equivalence using sentence embeddings with NLI gating. Four geometry features - dominant-cluster confidence T1, cluster count K, entropy H, and top-two share T2 - are read by the ordered decision rules R1 to R4, each of which binds one uncertainty type to one specialist agent. Certain items are answered directly. Epistemic items enter the retrieval branch, where gold context is injected and the model re-sampled; the delta test resolves the item only if confidence gains at least 0.10, and otherwise hands off to a human with the full audit trail attached. Aleatoric items bypass retrieval entirely and go to the clarification agent, which enumerates the competing interpretations. The two structural guarantees are marked: the aleatoric route contains no retrieval path at all, and every escalation the system emits carries enumerated, support-weighted options. . . . . | 19 |
| 5.1 | UTSR on the filtered benchmark across the three evaluated models (26 epistemic, 26 aleatoric, 15 certain controls). Aleatoric F1 falls monotonically with model capability ( $0.612 \rightarrow 0.490 \rightarrow 0.278$ ) while epistemic F1 rises ( $0.167 \rightarrow 0.375 \rightarrow 0.475$ ); this mirror-image pattern is the central cross-model finding and is analysed in Section 5.5. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 28 |
| 5.2 | UTSR against the three baselines on all three models, filtered benchmark. UTSR attains the highest macro F1 on every model and is the only system with non-zero aleatoric detection. B1’s zero wasted escalations are degenerate: it never escalates and consequently misses all 26 ambiguous items on every model. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | 29 |
| 5.3 | The three failure-mode counts on Llama 3.1 (8B), filtered benchmark; lower is better on all three. B3 exhibits the pathology it was constructed to represent, expending 63 futile tool calls on aleatoric items without resolving one. UTSR’s 26 accounted calls are the initial sampling rounds (Section 4.3.6); its aleatoric route executes zero retrieval by construction. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 30 |
| 5.4 | Failure-mode counts on Llama 3.2 (3B), filtered benchmark; lower is better. The ordering matches the mid-scale case of Figure 5.3: UTSR issues fewer wasted and fewer missed escalations than B2 (10 against 14, and 5 against 7), while B3 expends 48 futile tool calls. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 31 |
| 5.5 | Failure-mode counts on GPT-4o-mini, filtered benchmark; lower is better. UTSR again issues fewer wasted and fewer missed escalations than B2 (20 against 22, and 1 against 3). B3’s 67 futile tool calls are the campaign’s highest, reflecting the reflection loop’s inability to terminate on ambiguous items. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 32 |
| 5.6 | Routing distribution by ground-truth class on Llama 3.1 (8B), all systems, filtered benchmark. The certain controls fail conspicuously: 93% reach the human-handoff agent, a failure localised to the shared confidence estimator in Section 6.4 rather than to the routing rules. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | 33 |

|      |                                                                                                                                                                                                                                                                                                                                 |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.7  | Question-length distributions by class on the filtered benchmark. Means: epis-<br>temic 15.3 words, aleatoric 9.3 words. The distributions overlap in the 9 to 16<br>word band but each class retains a clearly separated mode; the residual six-word<br>difference is the disclosed confound discussed in Section 5.7. . . . . | 35 |
| 5.8  | Mean seconds per question and total wall-clock minutes on Llama 3.2 (3B), fil-<br>tered benchmark. UTSR’s overhead over the non-reflective baselines is 1.2×; it<br>completes 2.3× faster than B3. . . . .                                                                                                                      | 36 |
| 5.9  | Mean seconds per question and total wall-clock minutes on Llama 3.1 (8B), fil-<br>tered benchmark. UTSR pays a bounded overhead of 1.6× over the non-reflective<br>baselines and completes 1.8× faster than the reflection baseline B3. B1’s advan-<br>tage is degenerate: it never escalates and never retrieves. . . . .      | 36 |
| 5.10 | Mean seconds per question and total wall-clock minutes on GPT-4o-mini, filtered<br>benchmark. UTSR’s overhead over the non-reflective baselines is 1.5×; it com-<br>pletes 2.4× faster than B3. . . . .                                                                                                                         | 37 |

# List of Tables

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Uncertainty-quantification families for large language models. All four reduce uncertainty to a scalar magnitude and therefore cannot distinguish reducible from irreducible uncertainty, which is the gap UTSR addresses. . . . .                                                                                                                                                                                                                                                                 | 9  |
| 3.1 | Positioning of UTSR relative to the closest prior work. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                    | 17 |
| 4.1 | The three evaluation models, spanning a local 3B open-weight model to a proprietary cloud model. The same fixed configuration is applied to all three without per-model tuning. . . . .                                                                                                                                                                                                                                                                                                            | 22 |
| 4.2 | Dataset composition of the mixed benchmark. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                | 23 |
| 4.3 | The seven-criteria filter applied to both source datasets. The criteria are applied a priori as a construct-validity measure, motivated by the shortcut-solvability finding of Min et al. rather than by observed results. . . . .                                                                                                                                                                                                                                                                 | 24 |
| 4.4 | The three baseline systems. Each isolates one prevailing approach: B1 the no-escalation agent, B2 the magnitude-threshold router that dominates industrial practice, and B3 the type-blind agentic reflection loop. All four systems share the same sampler, clusterer and confidence estimator, so differences are attributable to the routing policy alone. . . . .                                                                                                                              | 25 |
| 4.5 | Final configuration, held fixed across all models and both dataset conditions. . . .                                                                                                                                                                                                                                                                                                                                                                                                               | 26 |
| 5.1 | Headline results, filtered benchmark. Macro F1 (E+A) averages the epistemic and aleatoric classes and is the primary metric; the three-class variant additionally averages the certain-control class and is reported for completeness. UTSR attains the highest value on both variants on every model. Waste rate is the share of accounted tool calls spent on aleatoric items; UTSR’s accounting includes its sampling rounds (Section 4.3.6), so the comparison is conservative against it. . . | 27 |
| 5.2 | UTSR per-class precision, recall, and F1 scores for each evaluated model on the filtered benchmark. Certain-control metrics are computed over the 15-item sanity set, while epistemic and aleatoric metrics are computed over the corresponding 26-item subsets. . . . .                                                                                                                                                                                                                           | 29 |
| 5.3 | Distribution of UTSR routing decisions by model and ground-truth (GT) class on the filtered benchmark. Values indicate the percentage of questions routed directly to an answer, through the epistemic retrieval/escalation path, or through the aleatoric clarification path. . . . .                                                                                                                                                                                                             | 33 |
| 5.4 | UTSR before and after the seven-criteria filter. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                           | 34 |

# List of Abbreviations

|          |                                          |
|----------|------------------------------------------|
| AI       | Artificial Intelligence                  |
| API      | Application Programming Interface        |
| AUQ      | Agentic Uncertainty Quantification       |
| B1       | Baseline System 1 - No Escalation        |
| B2       | Baseline System 2 - Fixed Threshold      |
| B3       | Baseline System 3 - AUQ-style Reflection |
| DAAO     | Difficulty-Aware Agentic Orchestration   |
| DAG      | Directed Acyclic Graph                   |
| F1       | F1 Score                                 |
| FM1      | Failure Mode 1 (Wasted Escalation)       |
| FM2      | Failure Mode 2 (Unresolvable Reflection) |
| GPT      | Generative Pre-trained Transformer       |
| GT       | Ground Truth                             |
| HF       | Hugging Face                             |
| LLM      | Large Language Model                     |
| MC       | Monte Carlo                              |
| MultiNLI | Multi-Genre Natural Language Inference   |
| NLI      | Natural Language Inference               |
| NLP      | Natural Language Processing              |
| QA       | Question Answering                       |
| RAG      | Retrieval-Augmented Generation           |
| ROI      | Return on Investment                     |
| RQ       | Research Question                        |
| SNLI     | Stanford Natural Language Inference      |
| SOP      | Standard Operating Procedure             |
| UQ       | Uncertainty Quantification               |
| UTSR     | Uncertainty-Type-Driven Subtask Routing  |

# 1 Introduction

Autonomous systems built on large language models are increasingly entrusted with work that unfolds over many steps, where the output of each step becomes the input of the next. In such systems the reliability of the whole is determined far less by the quality of any individual answer than by what happens when one component does not, in fact, know the answer. A conventional software component signals this condition by failing loudly; a language model does not. It produces a fluent and plausible response whether or not it has an adequate basis for one, and downstream components have no way of telling the difference. The question this thesis addresses is therefore not how to make agents more accurate, but how an orchestrator should behave when an agent is uncertain and, crucially, how the appropriate response depends on *what kind* of uncertainty is at play.

The remainder of this chapter develops that argument in stages. Section 1.1 establishes why uncertainty handling has become the critical reliability problem in agentic pipelines and introduces the distinction between epistemic and aleatoric uncertainty that the thesis builds upon. Section 1.2 translates this distinction into two concrete, measurable failure modes of current orchestration systems. Section 1.3 sketches the proposed mechanism, Uncertainty-Type-Driven Subtask Routing (UTSR), and the empirical test that makes the epistemic/aleatoric distinction operational at runtime. Section 1.4 states the research questions and the hypotheses derived from them, Section 1.5 summarises the contributions, and Section 1.6 delimits what this work does and does not claim. Section 1.7 closes with the structure of the document.

## 1.1 Motivation

Large language models (LLMs) have moved rapidly from single-turn text generators to the reasoning cores of agentic systems: software architectures in which an LLM observes a task, plans, calls external tools, incorporates the results, and iterates until a goal is reached [1]. Production-grade agentic AI systems now integrate multiple specialized agents with different underlying models, tool-augmented capabilities, orchestration logic, and external system interactions, forming dynamic pipelines capable of autonomous decision-making and action [2]. Such pipelines are increasingly deployed in structured workflow settings - compliance review, legal analysis, research pipelines, and customer escalation systems - in which a complex task is decomposed into subtasks that are processed by specialized agents arranged in a defined order, typically a directed acyclic graph (DAG).

As these systems take on consequential work, the question of reliability under uncertainty becomes central. An LLM agent, unlike a classical software component, does not fail loudly with an exception when it does not know something; it produces fluent, plausible text regardless of whether it has an adequate basis for its answer. This behaviour is the root cause of hallucination [3] and, in multi-step agentic settings, of a compounding phenomenon that recent work has called the spiral of hallucination: an early unfounded output is accepted by downstream agents as fact, contaminating every subsequent step of a long-horizon trajectory [4]. Uncertainty quantification (UQ) for LLMs has therefore become an active research area, producing methods based on self-consistency sampling [5], semantic entropy [6, 7], verbalised confidence [8], and self-evaluation [9].

Yet almost all of this work treats uncertainty as a scalar: a single confidence score or entropy value that is compared against a threshold. When the score crosses the threshold, the system takes a fixed remedial action - retry, reflect, retrieve, or escalate to a human - regardless of why the model is uncertain. This thesis argues that the scalar view is the wrong abstraction for pipeline orchestration, because uncertainty is not one phenomenon. Decision theory and Bayesian deep learning distinguish two fundamentally different kinds of uncertainty [10–12]:

- **Epistemic uncertainty** reflects a lack of knowledge on the part of the model. It is reducible: providing more relevant information - an additional document, a retrieval call, a tool invocation - can eliminate it. A multi-hop factual question whose supporting evidence the agent has not yet seen is epistemically uncertain.
- **Aleatoric uncertainty** reflects irreducible ambiguity or noise inherent in the task itself. It is not reducible by additional information: the question genuinely admits several valid answers because it is under-specified. Asking “when did England last win the World Cup?” is aleatorically uncertain, because the FIFA World Cup (1966) and the Cricket World Cup (2019) are both legitimate readings.

These two types of uncertainty demand opposite interventions. An epistemically uncertain agent should be routed to an information-gathering step; escalating it to a human wastes expensive reviewer attention on a case a retrieval agent could have handled. An aleatorically uncertain agent should not be given more tools or more reflection cycles - no amount of retrieval resolves a genuinely ambiguous question - it should surface the ambiguity explicitly and escalate with a structured set of valid interpretations. Current orchestration systems do neither: they apply the same fixed policy to both.

## 1.2 Problem Statement

Current multi-agent LLM pipelines treat uncertainty as a scalar quantity and respond to it with a fixed escalation path regardless of what causes the uncertainty. This produces two systematic and measurable failure modes, which this thesis formalises and evaluates separately:

1. **Failure Mode 1 (FM1) : Wasted Escalation.** Epistemically uncertain subtasks are escalated to human reviewers, who are expensive and slow, even though the uncertainty could have been resolved by routing the subtask to a retrieval agent or injecting an additional document. Human attention is consumed by cases that a well routed sub-agent could handle autonomously.
2. **Failure Mode 2 (FM2) : Unresolvable Reflection.** Aleatorically uncertain subtasks are routed into reflection loops or handed to additional agents that keep sampling and deliberating. Because the task is genuinely ambiguous, no amount of additional computation resolves it; the system burns tool calls and wall-clock time before timing out or emitting a forced answer that arbitrarily collapses a genuine ambiguity without ever surfacing it to the user.

Both failure modes stem from the same root cause: the orchestrator cannot see the type of the uncertainty, only its magnitude. The central problem addressed by this thesis is therefore: how

can an orchestrator, at runtime and without access to model weights, determine whether an agent’s uncertainty is epistemic or aleatoric, and exploit that determination to route each subtask to the intervention that can actually resolve it?

### 1.3 Proposed Approach: UTSR in Brief

This thesis introduces Uncertainty-Type-Driven Subtask Routing (UTSR), a training-free runtime mechanism that classifies agent uncertainty as either epistemic or aleatoric and routes each subtask accordingly. UTSR requires no fine-tuning, no access to logits or weights, and no labelled uncertainty data; it operates purely on the black-box sampling behaviour of the agent. Its core insight is that the two uncertainty types leave structurally different fingerprints in the distribution of an agent’s sampled outputs:

- When the agent is **epistemically** uncertain, its  $N$  sampled answers scatter across many semantically distinct clusters: the model is guessing among hypotheses because it lacks the information needed to converge.
- When the agent is **aleatorically** uncertain, its sampled answers polarise into a small number of stable, semantically opposed camps - typically two or three - because the model has correctly understood every valid interpretation of an ambiguous question and is distributing its probability mass among them.

UTSR operationalises this insight in a five-stage pipeline: (1) a subtask decomposer breaks a complex question into independent sub-questions; (2) a sampler draws  $N$  stochastic responses per subtask; (3) a semantic clusterer groups the responses using a natural language inference (NLI) cross-encoder, so that paraphrases merge and contradictions stay apart; (4) a pattern classifier reads the cluster geometry - dominant-cluster confidence, cluster count, distributional entropy, and top-two cluster dominance - and emits a typed label: CERTAIN, EPISTEMIC, or ALEATORIC; and (5) a router dispatches the subtask to the matching specialist: an answer agent for certain cases, a retrieval agent with a context-injection delta test for epistemic cases, and a clarification agent that enumerates valid interpretations - and issues zero retrieval calls - for aleatoric cases. A bounded two-attempt retrieval budget terminates loops that fail to converge, and a post-injection re-classification re-routes to clarification when the injected context reveals genuine ambiguity; a conservative aggregation policy composes sub-task verdicts back into a question-level decision.

The context-injection test deserves emphasis, because it converts the epistemic/aleatoric distinction from a philosophical definition into an empirically measurable property: if injecting relevant retrieved context into the prompt and re-sampling causes the answer distribution to collapse into a single dominant cluster (a large consistency gain  $\Delta$ ), the uncertainty was reducible and hence epistemic; if the distribution barely changes ( $\Delta$  below a threshold), the residual uncertainty is treated as irreducible. This behavioral operationalization sidesteps the definitional difficulties that recent position papers have identified in applying classical uncertainty taxonomies to interactive LLM settings (see Section 3.5).

### 1.4 Research Questions and Hypotheses

The thesis is organised around four research questions:

1. Can aleatoric and epistemic uncertainty be reliably distinguished in black-box LLM agents within a DAG pipeline - without access to model weights - using sample-distribution geometry combined with a context-injection test?
2. Does routing on uncertainty type rather than uncertainty magnitude reduce Failure Mode 1 (wasted human escalations on epistemic cases) relative to fixed-threshold escalation baselines?
3. Does typed routing reduce Failure Mode 2 (wasted tool calls and unresolved reflection cycles on aleatoric cases) relative to reflection-based agentic UQ baselines?
4. Is the mechanism robust across base models of different capability - that is, is the cluster-geometry signal a general property of LLM sampling rather than an artifact of one model's output entropy?

Corresponding to these questions, the following hypotheses are tested:

1. Cluster-geometry features computed over  $N = 8$  sampled answers, refined by a context-injection delta test, achieve substantially higher per-class F1 for epistemic/aleatoric discrimination than magnitude-only baselines, which by construction cannot separate the classes.
2. UTSR escalates significantly fewer epistemic cases to humans than a fixed-threshold router at matched or better answer quality, because reducible uncertainty is resolved by the retrieval branch before any escalation decision.
3. UTSR issues close to zero retrieval calls on aleatoric cases - by design the clarification branch performs none - whereas reflection-based baselines expend their full tool budget on such cases without converging; UTSR consequently exhibits lower unresolved-reflection rates and faster completion on aleatoric cases, at the cost of modest additional computation for the sampling and context-injection steps.
4. A single fixed configuration transfers across base models spanning the deployment spectrum: the ordering of systems on every headline metric is preserved from a small local model to a proprietary cloud model, because the epistemic and aleatoric fingerprints are a property of LLM sampling in general rather than of any one model.

## 1.5 Contributions

This thesis makes the following contributions:

1. **A training-free, black-box uncertainty-type classifier for LLM agents.** The classifier uses only sampled outputs, NLI-based semantic clustering, and cluster-geometry features (dominant-cluster confidence, cluster count, entropy, top-two dominance). To the best of my knowledge, no prior work uses cluster count and top-two share as the runtime mechanism for distinguishing epistemic from aleatoric uncertainty (Section 3.6).
2. **A context-injection delta test** that empirically verifies reducibility: rather than assuming a static definition of epistemic uncertainty, the system injects retrieved context, re-samples, and measures the consistency gain  $\Delta$ , treating uncertainty as epistemic only if the gain exceeds a threshold. This behavioural operationalisation is robust to the definitional breakdowns identified for interactive LLM settings.

3. **UTSR, a typed routing architecture for pipeline orchestration**, comprising a subtask decomposer, per-subtask uncertainty router, three specialist agents (answer, retrieval, clarification), a bounded retrieval loop with behavioural re-routing, type-tagged memory propagation, and a conservative aggregation policy in which aleatoric verdicts dominate. A two-signal reflection-cycle detector (stagnation and oscillation) is implemented and released alongside the system in the accompanying repository, but is not on the routing decision path in this evaluation.
4. **A dual failure-mode evaluation framework** that separately measures FM1 (wasted escalation of epistemic cases) and FM2 (unresolvable reflection on aleatoric cases), providing a more diagnostic view of pipeline reliability than existing single-metric evaluations.
5. **A joint benchmark construction** pairing HotpotQA (epistemic by construction: single-answer multi-hop questions) with AmbigNQ (aleatoric by construction: questions with  $\geq 2$  annotated valid answers), together with a fully reproducible open implementation on locally hosted models.

## 1.6 Scope and Delimitations

The thesis focuses on question answering as the sub-task type, because QA offers the cleanest publicly available ground truth for both uncertainty types. The system operates on black-box models served locally via Ollama; methods requiring logits, ensembles of independently trained models, or gradient access are out of scope. Human escalation is simulated: the evaluation measures whether an escalation would have been issued and whether it was warranted, not the downstream behaviour of actual human reviewers. Multilingual settings, multi-modal inputs, and adversarially ambiguous questions are left to future work. Finally, the evaluation reports classification quality, the two failure-mode counts, routing behavior, and latency; it does not measure downstream answer accuracy against gold answers, since the object of study is the routing decision rather than the answer itself.

## 1.7 Thesis Outline

Chapter 2 provides the theoretical background on uncertainty in machine learning and LLMs, semantic clustering with NLI, agentic architectures, and human-in-the-loop escalation. Chapter 3 reviews related work and positions UTSR in the literature, including a gap analysis against the closest prior systems. Chapter 4 presents the UTSR methodology and architecture in detail, with formal definitions of the classification rules and routing policies. Chapter 5 documents the experimental evaluation along with its results. Chapters 6 and 7 are reserved for discussion, conclusions and future works respectively.

## 2 Background

This chapter establishes the conceptual foundations of the thesis. Section 2.1 introduces the classical decomposition of predictive uncertainty. Section 2.2 surveys how uncertainty is estimated for large language models, with emphasis on black-box, sampling-based methods. Section 2.3 covers natural language inference and semantic clustering, the machinery UTSR uses to compare sampled answers. Section 2.4 reviews agentic LLM architectures, tool use, and reflection. Section 2.5 discusses human-in-the-loop escalation and the learning-to-defer literature. Section 2.6 introduces the two benchmark families - multi-hop QA and ambiguous QA - from which the evaluation datasets are drawn.

### 2.1 Two Kinds of Uncertainty

Predictive uncertainty is not a single quantity. The literature has long separated it into two components with different origins and more consequentially for this thesis - different remedies: uncertainty about what the model does not know, and uncertainty about what the question does not determine. This section sets out the classical form of that distinction (2.1.1), argues that its real content is a claim about which actions are worthwhile rather than merely a partition of variance (2.1.2), and reviews how the two components are estimated in conventional deep learning (2.1.3).

#### 2.1.1 The classical decomposition

The distinction between aleatoric and epistemic uncertainty long predates deep learning. In structural reliability engineering, Der Kiureghian and Ditlevsen [10] ask whether the distinction “matters” and answer affirmatively: aleatoric uncertainty is the variability intrinsic to the phenomenon being modelled, while epistemic uncertainty arises from the modeller’s incomplete knowledge and can, in principle, be reduced by gathering more data or refining the model. Crucially, they observe that the boundary between the two is model relative: what is irreducible noise under one modelling choice may become reducible knowledge under a richer one. This relativity foreshadows a difficulty that returns with force in the LLM setting (Section 3.5) and motivates the behavioural, test-based operationalisation adopted in this thesis. Kendall and Gal [11] imported the decomposition into Bayesian deep learning and demonstrated its practical value for computer vision. In their formulation, aleatoric uncertainty captures noise inherent in the observations - sensor noise, ambiguous pixels - and cannot be reduced even with more data, whereas epistemic uncertainty accounts for uncertainty in the model parameters, is largest in regions far from the training distribution, and can be explained away with more data. They further distinguish homoscedastic from heteroscedastic aleatoric noise and show that modelling both types jointly improves robustness. Hüllermeier and Waegeman [12] give the definitive machine-learning survey of the distinction, formalising it in terms of the learner’s hypothesis space: epistemic uncertainty is uncertainty about which hypothesis is correct, aleatoric uncertainty is the randomness that remains even under the true hypothesis. They also catalogue measures - variance decompositions, mutual information, credal sets - and warn that most practical estimators entangle the two components.

### 2.1.2 Why the decomposition matters for decision-making

The operational significance of the decomposition is that the two components license different actions. If uncertainty is epistemic, the rational response is information acquisition: query a database, run a retrieval step, ask a targeted question. If it is aleatoric, information acquisition is futile by construction - by definition, no observation reduces it - and the rational responses are instead decision-theoretic: hedge, present alternatives, or defer to an authority who can supply the missing specification (for example, the user who alone knows which “World Cup” they meant). Active learning exploits exactly this logic by acquiring labels where epistemic uncertainty is highest [13]. This thesis transplants the same logic from label acquisition into agent orchestration: the router acquires context where uncertainty is epistemic and surfaces alternatives where it is aleatoric.

### 2.1.3 Estimating the components in classical deep learning

In Bayesian neural networks, epistemic uncertainty is approximated through the posterior over weights, using Monte-Carlo dropout [14] or deep ensembles [15]; the mutual information between predictions and parameters isolates the epistemic component, while the expected entropy under fixed parameters isolates the aleatoric one [16]. These techniques, however, presuppose access to model internals - multiple stochastic forward passes with dropout, or a set of independently trained models. Neither assumption holds for the black-box, API- or Ollama-served LLMs considered in this thesis, which is why UTSR must reconstruct the decomposition purely from sampled outputs.

## 2.2 Uncertainty Quantification for Large Language Models

Estimating uncertainty for large language models is harder than for the classical models of the previous section, for two reasons. The natural confidence signal - sequence likelihood - is computed over surface forms and therefore conflates the model’s uncertainty about *what* is true with its uncertainty about *how* to phrase it. And the models of interest are typically served behind APIs or local runtimes that expose no logits at all, ruling out the internal-access techniques described above. The methods surveyed below respond by reconstructing uncertainty from externally observable behaviour: the agreement among repeated samples, the model’s own verbalised self-assessment, or the disagreement among several models or personas. What they share makes explicit, is that each reduces uncertainty to a scalar.

### 2.2.1 The calibration problem

Modern neural networks are systematically miscalibrated: their confidence scores overstate their accuracy [17], and pretrained transformers inherit the problem in NLP tasks [18]. For autoregressive LLMs the situation is aggravated by the token-level nature of the likelihood: sequence probability conflates the model’s uncertainty about what is true with its uncertainty about how to phrase it, and entropy computed over surface forms is dominated by paraphrase variation. [19] show that raw language-model probabilities are poor indicators of answer correctness in QA and that calibration requires task-specific adjustment. Recent work on the entropy calibration of language models further documents systematic divergence between token-level entropy and semantic-level confidence.

### 2.2.2 Sampling-based methods: self-consistency and semantic entropy

A family of black-box methods sidesteps the likelihood problem by sampling. Self-consistency [5] samples multiple chain-of-thought trajectories [20] at non-zero temperature and takes the majority answer; the size of the majority is a natural confidence proxy. SelfCheckGPT [21] detects hallucination by measuring agreement among sampled responses without any access to logits. The most influential refinement is semantic entropy [6, 7]: sampled answers are first grouped into semantic-equivalence classes using bidirectional NLI entailment, and entropy is computed over the classes rather than the strings. High semantic entropy - probability mass spread across many meaning-distinct clusters - signals that the model is confabulating; [7] show the method detects such confabulations across models and datasets, and the approach has been generalised to paragraph-length claims.

UTSR is a direct descendant of this line of work, but with a decisive shift of purpose. Semantic entropy compresses the clustered sample distribution into a single scalar used to flag unreliability. UTSR instead reads the shape of the clustered distribution - how many clusters, how dominant the largest, how the mass splits between the top two - as a typed signal, and uses that type to choose among qualitatively different downstream actions. Scalar semantic entropy cannot distinguish a model that scatters across seven clusters (epistemic guessing) from one that splits cleanly 60/40 between two interpretations (aleatoric ambiguity) when the two distributions happen to have similar entropy; the geometric features UTSR extracts are designed precisely to separate these regimes.

### 2.2.3 Verbalised and introspective confidence

A second family asks the model to report its own uncertainty. [22] teach GPT-3 to express calibrated uncertainty in words; [9] show that large models can predict the probability that their own answers are correct (“P(True)”) reasonably well in-distribution, but that this self-knowledge degrades under distribution shift. [8] benchmark verbalised confidence elicitation strategies and find persistent overconfidence, partially mitigated by sampling-and-aggregation hybrids. [23] find verbalised probabilities of RLHF-tuned models better calibrated than their token likelihoods. Verbalised confidence is attractive because it costs a single call, but it inherits the model’s blind spots: a model that misreads an ambiguous question as unambiguous will also misreport its confidence. UTSR therefore uses verbalisation only in the clarification agent’s output - for explaining an ambiguity - never as the classification signal.

### 2.2.4 Multi-model and multi-agent estimators

DiverseAgentEntropy [24] estimates uncertainty by having multiple agent personas interact and measuring the entropy of their aggregated answers, improving black-box hallucination detection and abstention over single-model sampling. Related work uses inter-agent critique to calibrate confidence. These approaches confirm that distributional structure over multiple generations carries reliable uncertainty signal, but they again reduce the structure to a scalar and do not attempt type separation.

### 2.2.5 Abstention and selective prediction

Selective prediction equips a model with a reject option [25]. For LLMs, abstention has been advocated both empirically - uncertainty-guided abstention measurably improves safety [26] - and programmatically, in position papers arguing that models should refuse or qualify rather than guess. [27] detect and mitigate hallucinations by validating low-confidence generations; [28] study how models handle questions they cannot know the answers to. [29] is particularly relevant to this thesis: studying ambiguous questions, they find that sampling-based repetition consistency predicts answerability better than likelihood-based confidence - evidence that sample-distribution structure specifically carries the aleatoric signal UTSR exploits. What none of these works provide is a downstream routing mechanism: abstention is a terminal action, whereas UTSR treats the abstain-versus-retrieve decision as the beginning of a typed workflow.

Table 2.1: Uncertainty-quantification families for large language models. All four reduce uncertainty to a scalar magnitude and therefore cannot distinguish reducible from irreducible uncertainty, which is the gap UTSR addresses.

| UQ family                        | Access required    | Output                   | Distinguishes E from A?  | Representative work |
|----------------------------------|--------------------|--------------------------|--------------------------|---------------------|
| Token likelihood / logit entropy | White-box(logits)  | Scalar                   | No                       | [17, 19]            |
| Ensembles / MC dropout           | White-box(weights) | Scalar decomposition     | Partially (in principle) | [13, 15]            |
| Self-consistency sampling        | Black-box          | Scalar (majority share)  | No                       | [5]                 |
| Semantic entropy                 | Black-box + NLI    | Scalar (cluster entropy) | No                       | [6, 7]              |
| Verbalised confidence            | Black-box          | Scalar (stated)          | No                       | [8, 22]             |
| Multi-agent entropy              | Black-box          | Scalar                   | No                       | [24]                |
| UTSR (this thesis)               | Black-box + NLI    | Typed label + route      | Yes (by design)          | -                   |

## 2.3 Natural Language Inference and Semantic Clustering

Reading the *shape* of a sample distribution rather than collapsing it to a scalar presupposes a way of deciding which sampled answers mean the same thing. This is a question of meaning rather than of string overlap: “Bing Crosby” and “The singer was Bing Crosby” are one answer, whereas “1966” and “2019” are two. Natural language inference supplies the machinery for that judgment, and clustering under semantic equivalence converts a bag of sampled strings into a distribution over distinct meanings - the object from which the geometric features used throughout this thesis are computed.

### 2.3.1 The Natural language Inference task

Natural language inference asks whether a hypothesis is entailed by, contradicts, or is neutral with respect to a premise. Large supervised corpora - SNLI [30] and MultiNLI [31] - enabled transformer-based NLI models of high accuracy, with DeBERTa [32] providing especially strong cross-encoder backbones. A cross-encoder NLI model jointly encodes the premise-hypothesis pair and outputs entailment/neutral/contradiction probabilities, trading throughput for accuracy relative to bi-encoder sentence embeddings - an appropriate trade for UTSR, which compares only  $N \leq 20$  short answers per subtask.

### 2.3.2 Semantic equivalence clustering of sampled answers

Following [6], two answers are considered semantically equivalent when each entails the other in context; clustering under this relation groups paraphrases (“Bing Crosby”, “The singer was Bing Crosby”) while separating genuinely different claims (“1966” vs “2019”). UTSR extends the merge criterion with three signals from the cross-encoder: a similarity/entailment score that must be high enough to merge, an entailment probability that must exceed a floor, and a contradiction probability that must stay below a ceiling - even mild contradiction blocks a merge. This asymmetric design reflects the cost structure of the downstream classifier: spuriously merging two distinct interpretations of an ambiguous question destroys the aleatoric fingerprint (a two-cluster split collapses into one “confident” cluster), whereas failing to merge two paraphrases merely inflates the cluster count slightly. The calibration history recorded in the released configuration (Appendix A.1) shows that permissive similarity thresholds (0.25, then 0.60) over-merged distinct interpretations before the final setting of 0.70 achieved clean separation.

### 2.3.3 From clusters to geometry

Once samples are clustered, the distribution  $p = (p_1, \dots, p_k)$  of sample mass over the  $k_k$  clusters supports several statistics used throughout this thesis: the confidence of the largest cluster  $\max_i p_i$ ; the cluster count  $k$ ; the Shannon entropy  $H(p) = -\sum p_i \log_2 p_i$  (computed in bits throughout, so that  $H \leq \log_2 N = 3$  for  $N = 8$ ); and the top-two dominance  $p(1) + p(2)$ , the combined share of the two largest clusters. Chapter 4 defines the classification rules over these features; intuitively, CERTAIN corresponds to one dominant cluster, ALEATORIC to high top-two dominance with moderate entropy (a 60/40 or 50/30/20 split among a few stable interpretations), and EPISTEMIC to many clusters with high entropy and no dominant pair.

## 2.4 LLM Agents, Tool Use, and Reflection

An uncertainty estimate is only useful if something in the system can act on it. This section reviews the actions available to an LLM agent - retrieving documents and calling tools, iterating in reflection loops, and the orchestration layer that arranges specialised agents into a pipeline - and the ways each fails when applied to the wrong kind of uncertainty. Two of these mechanisms map directly onto the branches of UTSR: retrieval is the appropriate intervention for reducible uncertainty, while reflection is the one that current systems misapply to uncertainty that no amount of deliberation can resolve.

### 2.4.1 Tool-augmented generation

Retrieval-augmented generation [33] grounds generation in retrieved documents and remains the canonical remedy for epistemic gaps: knowledge the model lacks is fetched at inference time. Toolformer [34] showed models can teach themselves when to call APIs; ReAct [1] interleaves reasoning traces with tool actions, establishing the observe-think-act loop underlying most modern agents. Self-RAG [35] trains a model to decide when retrieval is needed and to critique its own generations with reflection tokens - the closest tool-use analogue to typed routing, though its decision is still binary (retrieve or not) rather than typed (retrieve vs clarify vs answer).

### 2.4.2 Reflection and self-refinement

Reflexion [36] converts environment feedback into verbal self-reflections stored in episodic memory, improving sequential decision-making; Self-Refine [37] iterates generate-critique-revise loops with the same model. These mechanisms are powerful when errors are correctable - that is, when uncertainty is epistemic and the loop injects new information or catches a reasoning slip. On aleatoric tasks they exhibit precisely Failure Mode 2: the critique cannot identify a flaw, because there is none; the model oscillates between valid interpretations or restates its answer with cosmetic variation, and the loop terminates only on budget exhaustion. Recent analyses of self-correction confirm that intrinsic reflection without new external information often fails to improve, and can degrade, answer quality [38]. The cycle detector of Chapter 4 is designed around exactly the two observable signatures of this failure: stagnation (consistency gain  $\Delta$  below threshold across iterations) and oscillation (the dominant cluster flip-flopping between the same alternatives).

### 2.4.3 Multi-agent pipelines and orchestration

Frameworks such as AutoGen [2] and MetaGPT [39] compose specialised LLM agents into conversational or SOP-structured pipelines, encoding human workflow roles as agents. Orchestration research has begun to adapt pipeline structure to the task: difficulty-aware orchestration (DAAO) selects workflow depth, operators, and models per query based on predicted difficulty [40]. Two properties of this line matter for the present thesis. First, adaptation is driven by pre-execution difficulty estimates, not by uncertainty signals observed during execution. Second, whatever signal is used is again scalar - difficulty, not uncertainty type. UTSR is complementary: it operates at runtime, per subtask, and dispatches on type.

### 2.4.4 Error propagation in long-horizon pipelines

Because pipeline agents consume each other’s outputs, early errors compound: an unfounded claim accepted at step 1 infects every downstream step, the spiral of hallucination [4]. Uncertainty propagation through agent graphs is consequently an emerging topic, with recent work arguing that UQ modules must act not as passive sensors that merely diagnose risk but as active control signals that alter execution. UTSR embodies that argument: its type-tagged memory propagation (Section 4.2.3) stores every subtask output together with its uncertainty type and confidence, so downstream agents can adapt - most importantly, the conservative aggregation rule under which any aleatoric subtask verdict dominates the question-level decision, preventing a forced answer from silently absorbing an unresolved ambiguity.

## 2.5 Human-in-the-Loop Escalation and Learning to Defer

When neither retrieval nor reflection can resolve a case, the remaining option is to hand it to a person. Deferral has been studied in classical machine learning under the guiding principle of comparative advantage: a human should be called only for cases the model cannot resolve itself, because reviewer attention is the scarcest resource in the system. This section reviews that literature, the evidence that oversight cost dominates the economics of deployed agent systems, and the question of what an escalation should actually contain - the standard against which the clarification output of UTSR is designed.

### 2.5.1 Learning to defer

The learning-to-defer framework [41] extends rejection learning to settings where the fallback is a human decision-maker with their own error profile and costs; [42] give consistent surrogate losses for jointly learning the predictor and the deferral policy. The framework’s central lesson is that when to defer must reflect the comparative advantage of human and machine - deferring cases the machine could resolve wastes the scarce human resource. FM1 is exactly this waste, instantiated in LLM pipelines: fixed-threshold escalation defers epistemic cases whose comparative advantage lies with a retrieval agent, not a human.

### 2.5.2 The cost of human oversight in agentic systems

Position work on agentic ROI argues that the real barrier to LLM-agent usability is the total return on investment, which includes human oversight cost, time, and effort required to operate and maintain agents; continuous human feedback is framed explicitly as a cost component to be optimised. Domain studies of human-in-the-loop agentic AI in financial services echo the point: compliance settings demand human control, but scalable designs must call humans only when necessary, and every escalation must be auditable. Hybrid human-AI collaboration research likewise emphasises the tension between embedding continuous human input and allowing agents to act autonomously. UTSR addresses the auditability requirement directly: every escalation carries a typed reason - an aleatoric escalation names the ambiguity and enumerates the valid interpretations with the specific missing specification identified, so a reviewer can resolve it in seconds rather than reconstructing the problem from an opaque “needs review” flag.

### 2.5.3 What a good escalation looks like

The clarification agent of Chapter 4 is designed to the standard articulated in recent position work on UQ for agents: when an agent detects that it lacks knowledge it should retrieve; when aleatoric uncertainty remains, it should communicate not merely a number but why it is uncertain, what options exist, and what further information would resolve the ambiguity. Concretely, on a genuinely ambiguous legal clause the desired output is of the form “this clause is genuinely ambiguous; under interpretation A, under interpretation B; the missing specification is the governing jurisdiction” - a structured set of valid interpretations rather than a confused single answer.

## 2.6 Benchmark Foundations: Multi-hop and Ambiguous QA

Evaluating a classifier of uncertainty *type* requires items whose true type is already known. Annotating questions as epistemic or aleatoric by hand would be laborious and, worse, circular: the annotator’s judgment of what makes a question ambiguous is precisely what the mechanism is supposed to determine. The alternative pursued here is to exploit datasets in which the uncertainty type follows from how the data was constructed rather than from any additional labelling. The two question-answering families introduced below - multi-hop QA and ambiguous QA - provide exactly this, one for each type.

### 2.6.1 HotpotQA: epistemic uncertainty by construction

HotpotQA [43] contains 113k Wikipedia-based question-answer pairs that require multi-hop reasoning: answering demands combining facts from multiple supporting paragraphs, with sentence-level supporting-fact annotations and, in the distractor configuration, eight distractor paragraphs accompanying the two gold ones. Each question has a single correct answer; a model’s uncertainty on such questions stems from missing knowledge or missing evidence and is therefore epistemic by construction - supplying the gold supporting paragraphs (context injection) resolves it. This inherent labelling, requiring no manual annotation by the experimenter, is the property that makes HotpotQA the epistemic half of the benchmark (Section 4.3.2).

### 2.6.2 AmbigQA / AmbigNQ: aleatoric uncertainty by construction

AmbigQA [44] re-annotates Natural Questions [45] and finds that over half of naturally occurring information-seeking questions are ambiguous - temporally (“who is the president?”), referentially (“where is Springfield?”), or through event multiplicity (“when did England last win the World Cup?”). The AmbigNQ resource annotates, for each question, the set of distinct valid answers paired with the disambiguated question each answers. Filtering to questions with at least two distinct valid answers yields items whose uncertainty is aleatoric by construction: the ambiguity is a property of the question, and no retrieval collapses the answer set. Follow-up work uses AmbigQA precisely as an aleatoric testbed, observing that highly ambiguous questions should incur high aleatoric uncertainty, and surveys note that mainstream UQ benchmarks lack aleatoric items - a gap the joint HotpotQA + AmbigNQ design of this thesis is built to fill.

### 2.6.3 Why the pairing is clean

Three properties make the pairing suitable. First, the ground-truth uncertainty type is inherent to dataset design rather than manually assigned: HotpotQA questions require multi-hop evidence (epistemic), AmbigNQ questions have multiple annotated valid answers (aleatoric). Second, both datasets are established, heavily cited benchmarks, so results are interpretable against a large literature. Third, the context regimes align with the routing semantics: HotpotQA supplies supporting documents that the retrieval branch can inject, while AmbigNQ items are deliberately given no context - no context helps, which is the point. Alternative and supplementary datasets (TriviaQA, Natural Questions, PopQA on the epistemic side; SituatedQA and QAMPARI on the aleatoric side) are discussed as robustness extensions in Section 7.5.

## 3 Related Work

This chapter positions UTSR against five adjacent research threads: (i) scalar uncertainty quantification and abstention for LLMs; (ii) ambiguity detection and clarification in question answering; (iii) uncertainty-aware agents and agentic UQ frameworks; (iv) orchestration and routing in multi-agent pipelines; and (v) conceptual critiques of the epistemic/aleatoric taxonomy in interactive settings. Section 3.6 consolidates the gap analysis.

### 3.1 Scalar Uncertainty Quantification and Abstention

The sampling-based UQ line surveyed in Section 2.2 - self-consistency [5], SelfCheckGPT [21], semantic entropy [6, 7], verbalised confidence [8, 9, 22], and multi-agent entropy [24] - establishes that black-box distributional signals reliably track reliability. Comprehensive surveys of LLM UQ [46] organise these methods and their applications to hallucination detection and selective answering. Two consistent limitations recur. First, the output is a scalar, so the remedial action must be chosen by a fixed policy independent of the uncertainty’s cause. Second - and documented explicitly by a recent survey of UQ evaluation practice [47] - there is a prevailing tendency to evaluate on established QA benchmarks that lack aleatoric items entirely, and UQ methods designed only for epistemic uncertainty measurably underperform on tasks containing aleatoric uncertainty. UTSR responds to both limitations: its output is a typed label bound to a routing action, and its benchmark deliberately balances the two uncertainty types.

Within the abstention literature, [26] show uncertainty-guided abstention improves safety; [27] validate low-confidence spans to mitigate hallucination; [28] study known-unknown questions; and [29] show that on ambiguous questions, sampling repetition is the best predictor of answerability. Abstention, however, is a terminal act. UTSR generalises it into a three-way routing decision in which “abstain” becomes “escalate with enumerated interpretations,” and only for the aleatoric type.

### 3.2 Ambiguity in Question Answering

AmbigQA [44] established ambiguity as a first-class phenomenon in open-domain QA and introduced the task of predicting disambiguated question-answer pairs. Subsequent work asks clarification questions in dialogue, detects when clarification is needed, and selectively answers ambiguous questions [29]. SituatedQA [48] isolates temporal and geographic context-dependence; QAMPARI targets questions with many valid answers. This literature develops sophisticated detectors of ambiguity in the question text, typically as supervised classifiers or prompted judgments. UTSR differs in mechanism and in scope of deployment: it does not inspect the question at all, but the shape of the agent’s own answer distribution - a signal available for any subtask in any pipeline, including subtasks that are not natural-language questions in the benchmark sense - and it embeds the detector in an orchestration loop where the detection outcome changes tool usage and escalation, not just the final answer format.

### 3.3 Uncertainty-Aware Agents and Agentic UQ

A rapidly growing thread makes agents uncertainty-aware. Self-RAG [35] trains retrieval-decision and self-critique tokens; adaptive-RAG variants trigger retrieval only when confidence is low. Closest in spirit to this thesis are two 2025 works identified during the literature study. The first is a position paper [49], which argues in words for precisely the behaviour UTSR implements: when an agent detects it lacks knowledge it should retrieve, and when aleatoric uncertainty remains it should communicate not just a number but why it is uncertain, what options exist, and what further information could help. Being a position paper, it proposes no mechanism, no classifier, and no evaluation; this thesis can be read as the constructive counterpart that supplies all three. The second is the Dual-Process Agentic UQ (AUQ) framework, which addresses the spiral of hallucination - early epistemic errors propagating irreversibly across long-horizon tasks - by transforming verbalised uncertainty into active bi-directional control signals, observing that existing UQ acts as a passive sensor and that self-reflection suffers from continuous or aimless corrections. AUQ shares UTSR’s diagnosis (passive scalar UQ is insufficient; unguided reflection wastes effort) but not its remedy: AUQ’s control signal remains type-blind, driving a reflection loop whose intensity scales with magnitude. For exactly this reason a faithful re-implementation of the AUQ reflection pattern serves as baseline B3 in Chapter 6, where it is expected to exhibit Failure Mode 2 on aleatoric items.

The only prior work found that explicitly decomposes epistemic and aleatoric uncertainty in a multi-agent context is “Epistemic Gain, Aleatoric Cost” [50], which performs the decomposition within a symmetric debate format for mathematical reasoning. Its decomposition is analytic - quantifying how debate rounds increase epistemic gain at aleatoric cost - and it does not use uncertainty type as a routing signal in structured pipeline workflows. Its existence confirms the decomposition is tractable for LLM agents while leaving the routing question, the subject of this thesis, open.

### 3.4 Orchestration, Routing, and Human Escalation in Pipelines

Multi-agent frameworks (AutoGen, [2]; MetaGPT, [39]) provide the structural substrate - role-specialised agents in SOP-like or conversational graphs - but route by static graph topology. Difficulty-aware Agentic orchestration (DAAO; [40]) adapts workflow depth, operator selection, and model assignment per query, but from difficulty predicted before execution, not from uncertainty types observed at runtime during the task. Escalation-focused designs, such as fixed multi-level DAG escalation policies route all uncertain cases through the same review levels regardless of uncertainty source, and report human-review reductions relative to escalate-everything policies; because the escalation trigger is scalar, epistemic and aleatoric cases traverse identical paths, so such systems retain FM1 in the residual escalations and cannot address FM2 at all. UTSR should match or improve on their human-review reduction while additionally eliminating the unresolvable-reflection problem entirely, since aleatoric cases bypass retrieval and reflection by construction. The learning-to-defer literature [41, 42] supplies the decision-theoretic vocabulary but assumes a single predictor and a human - not a pipeline with heterogeneous machine remedies (retrieval vs clarification) interposed before the human.

### 3.5 Critiques of the Epistemic/Aleatoric Taxonomy for LLMs

A conceptual caveat must be acknowledged. Recent analysis argues that the classical definitions of epistemic and aleatoric uncertainty break down in interactive LLM settings: whether uncertainty is “reducible” depends on what interventions are admissible (asking the user a question reduces uncertainty that retrieval cannot), echoing Der Kiureghian and Ditlevsen’s [10] observation that the boundary is model-relative. This thesis takes a complementary, deliberately pragmatic position: instead of relying on static definitions, UTSR operationalises the distinction through a behavioural test relative to a fixed intervention class. Uncertainty is treated as epistemic exactly when the admissible intervention - context injection via the retrieval agent - measurably reduces it (consistency gain  $\Delta \geq \theta\Delta$ ), and as aleatoric when it does not. Under this operational definition the classifier’s claims are falsifiable within the system’s own action space, and the philosophical dispute about edge cases is converted into a measurable property of the pipeline.

### 3.6 Gap Analysis and Positioning

Synthesising the threads above, the literature contains: scalar detectors of unreliability (3.1); question-side ambiguity detectors (3.2); agents that decide whether to retrieve or reflect on a scalar signal (3.3); orchestrators that adapt structure to predicted difficulty (3.4); and one decomposition of E/A uncertainty inside a debate protocol (3.3). A systematic search conducted for this thesis found *no work that uses uncertainty type* - rather than uncertainty magnitude - as a live routing signal in a DAG-structured multi-agent pipeline, dispatching different specialist agent roles depending on whether the current uncertainty is reducible or irreducible. More specifically, no paper uses cluster count and top-two cluster share over NLI-clustered samples as the runtime mechanism to distinguish epistemic from aleatoric uncertainty; and while individual papers use AmbigQA as an aleatoric testbed, none combines it with HotpotQA as the epistemic counterpart in a joint routing evaluation. This thesis occupies exactly that intersection, as summarised in Table 3.1.

Table 3.1: Positioning of UTSR relative to the closest prior work.

| Work                                   | Signal                                                         | E/A separation | Routing action                                           | Pipeline setting           | Evaluation of both failure modes |
|----------------------------------------|----------------------------------------------------------------|----------------|----------------------------------------------------------|----------------------------|----------------------------------|
| Semantic entropy                       | Cluster entropy (scalar)                                       | No             | Flag/abstain                                             | Single model               | No                               |
| Uncertainty-guided abstention [26, 29] | Scalar confidence                                              | No             | Abstain (terminal)                                       | Single model               | No                               |
| Self-RAG [35]                          | Learned critique tokens                                        | No             | Retrieve or not                                          | Single agent               | No                               |
| Agentic reflection / AUQ [36, 38]      | Verbalised uncertainty magnitude                               | No             | Reflect and retry; intensity scales with magnitude       | Single agent, long horizon | No (generates FM2)               |
| DAAO [40]                              | Predicted difficulty, pre-execution                            | No             | Select workflow depth, operator, model                   | Multi-agent DAG            | No                               |
| Qiao et al. [50]                       | Bayesian E/A decomposition                                     | <b>Yes</b>     | RL training objective; no run-time routing               | Symmetric debate, maths    | No                               |
| <b>UTSR (this thesis)</b>              | Cluster geometry $(c, k, d, H)$ + context-injection delta test | <b>Yes</b>     | Typed dispatch: answer / retrieve / clarify and escalate | Multi-agent DAG pipeline   | <b>Yes</b>                       |

# 4 Methodology: The UTSR System Design and Experimental Setup

This chapter presents the design of the Uncertainty-Type-Driven Subtask Routing (UTSR) system and the experimental setup used to evaluate it. Section 4.1 restates the research questions. Section 4.2 describes the system design: the uncertainty-type classifier built on sampling, semantic clustering, and cluster geometry, and the routing layer of specialist agents supervised by a context-injection delta test and a bounded retrieval loop. In addition, this chapter also documents the concrete implementation of UTSR and of the experimental harness in Section 4.3 which specifies the experimental setup: the three evaluation models, the benchmark construction including the multi-criteria dataset filter and the certain-control set, the confound audit, the baseline systems, the evaluation metrics, and the experimental protocol.

## 4.1 Research Questions

The evaluation is organised around four questions. RQ1: Can epistemic and aleatoric uncertainty be distinguished in black-box LLM agents at runtime, without access to model weights, using the geometry of the agent’s sampled answer distribution refined by a context-injection test? RQ2: Does routing on uncertainty type rather than uncertainty magnitude reduce wasted human escalations on epistemic cases (Failure Mode 1, FM1) relative to fixed-threshold escalation? RQ3: Does typed routing reduce wasted tool calls and unresolvable reflection on aleatoric cases (Failure Mode 2, FM2) relative to reflection-based agentic baselines? RQ4: Is the mechanism robust across base models of different capability, i.e., is the cluster-geometry signal a property of LLM sampling in general rather than an artifact of one model?

RQ4 is answered by a cross-model campaign rather than a threshold sweep: rather than sweeping thresholds around one model’s operating point, the final experimental campaign holds the configuration fixed and varies the base model across three systems spanning the deployment spectrum, from a small local open-weight model to a proprietary cloud model. If the epistemic/aleatoric fingerprints in cluster geometry are a fundamental property of LLM sampling, the mechanism should transfer; if they are an artifact of one model’s output entropy, the cross-model comparison will expose it. As Section 5.5 shows, this design choice proved decisive: it surfaced the most important empirical finding of the thesis.

## 4.2 System Design

The design of UTSR follows from a single structural hypothesis about how the two uncertainty types manifest in sampled output, and everything else in the system is machinery for detecting that manifestation and acting on it. This section states the hypothesis, then specifies the two components built on top of it: the training-free classifier that samples the agent, clusters the responses semantically, and reads the resulting cluster geometry as a typed label; and the routing layer that binds each label to a specialist agent, supervised by a context-injection delta test and a bounded retrieval loop. Fig. 4.1 displays the entire UTSR pipeline in detail.

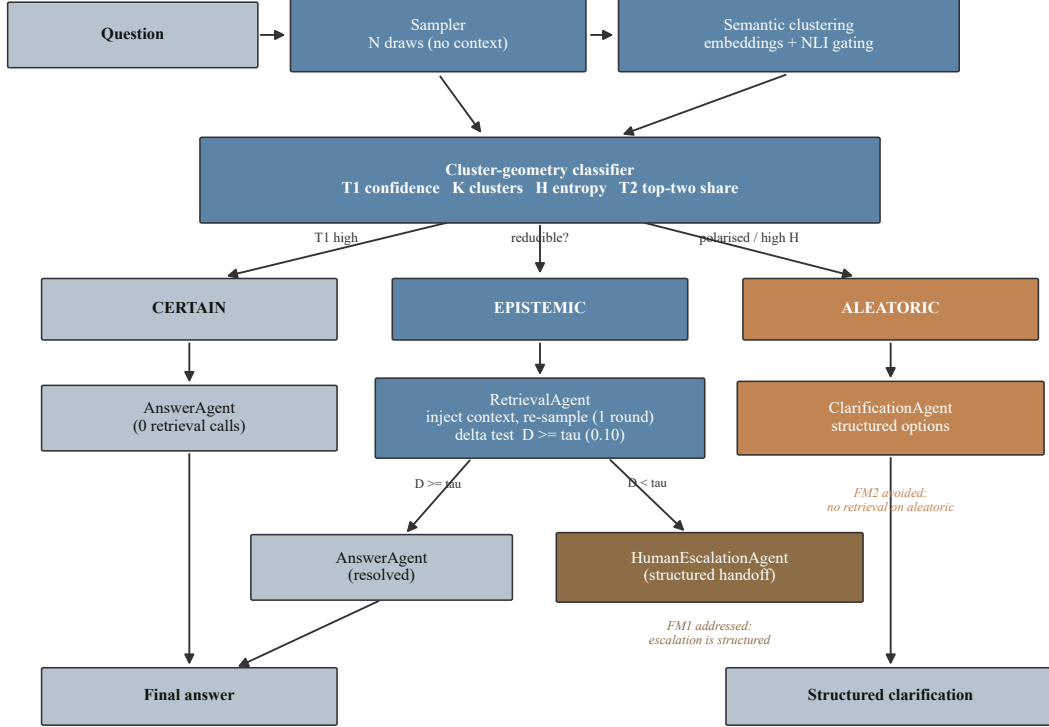


Figure 4.1: The UTSR pipeline. A question is sampled  $N$  times without context and the samples are clustered under semantic equivalence using sentence embeddings with NLI gating. Four geometry features - dominant-cluster confidence  $T1$ , cluster count  $K$ , entropy  $H$ , and top-two share  $T2$  - are read by the ordered decision rules  $R1$  to  $R4$ , each of which binds one uncertainty type to one specialist agent. Certain items are answered directly. Epistemic items enter the retrieval branch, where gold context is injected and the model re-sampled; the delta test resolves the item only if confidence gains at least 0.10, and otherwise hands off to a human with the full audit trail attached. Aleatoric items bypass retrieval entirely and go to the clarification agent, which enumerates the competing interpretations. The two structural guarantees are marked: the aleatoric route contains no retrieval path at all, and every escalation the system emits carries enumerated, support-weighted options.

#### 4.2.1 Design rationale

UTSR rests on a single structural hypothesis: the two classical uncertainty types [10–12] leave different fingerprints in the distribution of an agent’s stochastically sampled answers. When the agent is epistemically uncertain - it lacks knowledge - its  $N$  samples scatter across many semantically distinct clusters, because the model is guessing among hypotheses. When it is aleatorically uncertain - the question itself is ambiguous - its samples polarise into a small number of stable, mutually contradicting camps, one per valid reading, because the model has understood the interpretations and is distributing probability mass among them. When the agent is certain, the samples converge on one dominant cluster. Scalar uncertainty measures such as semantic entropy [6, 7] compress this distribution into a single number; UTSR instead reads its shape and binds each shape to a different downstream action.

### 4.2.2 The uncertainty-type classifier

The classifier is training-free and operates in three stages on each incoming question.

**Sampling:** The agent is queried up to  $N = 8$  times with an identical prompt at temperature  $\tau = 0.60$ . The temperature is chosen so that the sample distribution reveals uncertainty: near-zero temperature collapses all samples onto the mode and makes every question look certain, while very high temperature makes even well-known facts look scattered. The sample count of eight is the smallest at which the three target geometries remain distinguishable: at least six of eight samples in one cluster clears the certainty bar; a 5/3 or 4/4 split is unambiguously polarised rather than sampling noise; and five or more clusters over eight samples - most necessarily singletons - is unambiguous scatter. Early stopping is permitted in exactly one direction: after a minimum of five samples the running semantic confidence is re-checked every second sample, and sampling terminates early only when confidence already clears the certainty bar. Stopping on disagreement is never permitted, because it truncates precisely the distributions whose shape the classifier needs

**Semantic clustering:** Samples are grouped into semantic-equivalence clusters, following the semantic-equivalence paradigm of [6]. Each incoming sample is compared against existing cluster centroids using sentence embeddings (all-mpnet-base-v2; [51]) and against cluster representatives using a natural language inference cross-encoder (nli-deberta-v3-small; [32]). A merge requires the conjunction of three conditions: cosine similarity of at least 0.70, entailment probability of at least 0.55, and contradiction probability of at most 0.35. The conjunction is deliberately strict: the dominant failure of permissive clustering is over-merging, which collapses the two-camp aleatoric fingerprint into a single confident-looking cluster, whereas under-merging merely inflates the cluster count slightly. The output is the cluster partition sorted by size, from which four geometry features are computed: the largest-cluster confidence  $c$ , the cluster count  $k$ , the top-two dominance  $d$ , and the Shannon entropy  $H$  of the cluster mass distribution.

**Pattern classification:** The geometry is mapped to one of three labels by ordered decision rules. **R1 (CERTAIN):**  $c \geq 0.65$ , i.e., at least six of eight samples in one cluster. **R2 (ALEATORIC):** structured disagreement, detected by any four signals - polarisation with bounded entropy ( $d \geq 0.60$  and  $H \leq 2.0$ ); a clean binary or ternary split ( $k \in 2, 3$  with  $d \geq 0.75$  and  $c$  below the certainty bar); a compact polarised set ( $k \leq 4$  with  $d \geq 0.60$ ); or a small, moderately concentrated interpretation set ( $2 \leq k \leq 5$  with  $d \geq 0.50$  and  $c$  below the certainty bar), added after inspection showed genuinely ambiguous questions frequently produce a handful of distinct readings with only moderate polarisation. **R3 (EPISTEMIC):** scattered disagreement,  $H \geq 2.5$  or  $k \geq 5$ . **R4 (fallback):** any remaining borderline geometry defaults conservatively to ALEATORIC, on the cost argument that a wasted clarification costs one human glance while a wasted retrieval loop costs the full tool budget. The certainty bar of 0.65 was calibrated down from an initial 0.80 after observing that small local models rarely produce 80% semantic agreement even on questions they answer correctly and consistently. The rules are evaluated in order, so a geometry satisfying both R2 and R3 (for example  $k = 5$  with  $d \geq 0.50$ ) is labelled aleatoric; this ordering implements the conservative preference stated in R4.

### 4.2.3 Routing, specialist agents, and loop supervision

Each label is bound to a specialist agent with a defined contract. CERTAIN routes to the **Answer-Agent**, which emits the representative of the largest cluster at zero additional cost. ALEATORIC

routes to the **ClarificationAgent**, which issues zero retrieval calls - the route contains no tool-calling path at all - and packages a structured escalation: up to three enumerated interpretation options, each carrying the corresponding cluster representative as its candidate answer together with its sample-support count and fraction, plus a statement of the ambiguity and the action required of the reviewer. EPISTEMIC routes to the **RetrievalAgent**, which fetches context and hands control to the context-injection delta test: the agent is re-sampled with the retrieved context injected, re-clustered, and the consistency gain  $\Delta$  (post-injection confidence minus pre-injection confidence) is measured. A gain of  $\Delta \geq 0.10$  confirms the uncertainty was reducible - it was genuinely epistemic - and the converged answer is emitted; a post-injection geometry that classifies aleatoric re-routes the question to clarification, catching geometric misclassifications behaviourally; and exhaustion of the two-attempt budget hands the question to the **HumanEscalationAgent**, which emits a structured handoff containing the same enumerated candidate options plus the complete audit trail of retrieval attempts and  $\Delta$  values. Two loop-termination behaviours follow from this design. Stagnation - cumulative confidence gain below the  $\Delta$  threshold across the attempt budget - terminates the loop and hands off to the escalation agent. Oscillation between competing readings surfaces behaviourally: when a post-injection geometry re-classifies as aleatoric, the question is re-routed to clarification rather than pursued further. A standalone two-signal reflection-cycle detector implementing both criteria explicitly (stagnation and sign-reversal oscillation) is released with the system in the accompanying repository, but is not on the routing decision path in this evaluation; the bounded budget and the behavioural re-route together provide the same guarantees at the scale studied here. All specialist agents are deterministic, rule-based components containing no LLM calls of their own; the only LLM inference in the pipeline is the sampling itself, so the routing overhead beyond sampling is limited to embedding and NLI computation. Every subtask outcome is written to a type-tagged pipeline memory (value, uncertainty type, confidence, tool calls, resolved flag), and a conservative aggregation policy propagates aleatoric verdicts upward. A subtask decomposer is available for multi-hop production pipelines but runs in pass-through mode for this evaluation, since both benchmark datasets are constructed to be answered as single units.

Two structural guarantees follow from the design rather than from tuning. First, an item classified aleatoric can never trigger a retrieval call (the FM2 guarantee is a property of the code path). Second, every escalation the system emits - whether via clarification or via last-resort human handoff - is structured with enumerated, support-weighted options (the FM1-quality guarantee), converting escalation from an opaque “needs review” flag into a decision a reviewer can resolve in a single message.

## 4.3 Experimental Setup

Testing the four research questions requires a setup that can separate the mechanism’s behaviour from the properties of any single model or dataset. Three commitments follow from this. The configuration is held fixed and run unchanged across base models spanning the deployment spectrum, so that a signal specific to one model’s output entropy would be exposed rather than hidden. The benchmark draws its ground-truth uncertainty types from dataset construction rather than manual annotation, and is audited for confounds that could produce the expected separation for the wrong reason. And the baselines instantiate the two failure modes directly, so that each research ques-

tion maps onto a measurable comparison. The subsections below specify these elements in turn, together with the metrics and the experimental protocol.

### 4.3.1 Evaluation models

The mechanism claims to be model-agnostic: it touches no logits, weights, or gradients, and operates purely on sampled text. To test this claim the final campaign runs the identical configuration on three base models chosen to span the deployment spectrum which is detailed in Table 4.1.

Table 4.1: The three evaluation models, spanning a local 3B open-weight model to a proprietary cloud model. The same fixed configuration is applied to all three without per-model tuning.

| Model          | Scale / access          | Serving          | Deployment archetype                       |
|----------------|-------------------------|------------------|--------------------------------------------|
| Llama 3.2 (3B) | Small, open weights     | Local via Ollama | Resource-constrained on-premise deployment |
| Llama 3.1 (8B) | Medium, open weights    | Local via Ollama | Mid-range local deployment                 |
| GPT-4o-mini    | Proprietary cloud model | OpenAI API       | Cloud-based production systems             |

The pairing of a weak local model with a strong commercial model maximises the capability gap and thereby provides the strongest test of generalisability: the two ends differ in architecture, training methodology, parameter count, and deployment mode, so behaviour that is consistent across them cannot be an artifact of a single model family. All classifier thresholds were calibrated once, on the 3B model, and then held fixed across all three models - a deliberately hard test whose consequences are analysed in Section 5.5.

### 4.3.2 Benchmark construction

The benchmark pairs two datasets whose ground-truth uncertainty type is inherent to their construction, requiring no manual uncertainty annotation. Epistemic items are drawn from the validation split of HotpotQA in the distractor configuration [43]: multi-hop questions with a single gold answer and supporting Wikipedia paragraphs, which serve as the injectable context for the retrieval branch - simulating a retriever with guaranteed recall so that the delta test measures the classifier and router rather than retriever quality. Aleatoric items are drawn from the validation split of AmbigNQ [44], which re-annotates Natural Questions [45] with the set of distinct valid answers per question; only items with at least two distinct valid answers are retained, and their context is deliberately empty - no context helps, which is the point. A third, small certain-control set of 15 trivial single-hop factual questions with unambiguous answers (geography, basic science, arithmetic, general knowledge) closes the missing CERTAIN class: it is a sanity control rather than a powered comparison, verifying that the pipeline routes obviously easy cases to the direct-answer path. Its outcomes are reported as raw proportions; it is excluded from the powered epistemic-versus-aleatoric metrics.

The main benchmark uses 26 epistemic + 26 aleatoric items, shuffled with a fixed seed. The class size follows a standard power calculation: for a two-sided comparison at  $\alpha = 0.05$  with

80% power to detect a large effect ( $d = 0.80$ ), the required sample is 26 per group [52]. The expected effects here are large by construction - the baselines possess no aleatoric route at all, so the primary contrast is between substantial and zero detection - making  $n = 26$  the smallest sample that still yields a clear, non-random signal while keeping the experiment feasible across three model backends: the clustering pipeline invokes the NLI cross-encoder for every candidate merge in every sampling round, and a full four-system run over the benchmark already costs one and a half to over five hours of wall-clock time per model (Section 5.8). The resulting benchmark composition is summarized in Table 4.2.

Table 4.2: Dataset composition of the mixed benchmark.

| Property                  | HotpotQA (epistemic)                                                                                                     | AmbigNQ (aleatoric)                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| HF dataset / config       | hotpotqa/hotpot_qa, distractor                                                                                           | sewon/ambig_qa, light                                                                |
| Split (size)              | Validation (7,405)                                                                                                       | Validation (2,002)                                                                   |
| Items used                | 26 ( $N_{\text{EPISTEMIC\_SAMPLES}}$ )                                                                                   | 26 ( $N_{\text{ALEATORIC\_SAMPLES}}$ , $\geq 2$ distinct answers)                    |
| Ground-truth label source | Single answer + required multi-hop evidence (by construction)                                                            | Annotated multiple valid answers (by construction)                                   |
| Context supplied          | Gold supporting paragraphs (injectable)                                                                                  | Empty (no context helps, by design)                                                  |
| Example                   | “College Humor” is a 1933 film that starred what American singer with a warm bass-baritone voice? → <b>Bing Crosby</b> . | “When was the last time England won the World Cup?” → {1966 (FIFA), 2019 (Cricket)}. |

### 4.3.3 The dataset filtering protocol

[53] showed that a substantial fraction of HotpotQA questions is solvable through single-hop shortcuts, which contaminates the epistemic ground truth: a shortcut-solvable question does not actually require the missing evidence, so a model may answer it consistently from parametric knowledge and the item ceases to be epistemically uncertain by construction. The shortcut-solvability finding of [52] motivated a multi-criteria filter applied to the HotpotQA side of the final benchmark which are detailed in Table 4.3.

Criteria 1-6 strengthen the epistemic class directly; criterion 7 addresses a separate threat, examined next. The aleatoric side is otherwise unchanged, since AmbigNQ’s  $\geq 2$  - answers annotation is not affected by shortcut contamination.

### 4.3.4 The length-confound audit

The thesis claim concerns the model’s sampling behaviour under uncertainty, not surface features of the question text. If HotpotQA questions were systematically long and AmbigNQ questions systematically short, a classifier could achieve apparent type separation by responding to length alone. A dedicated audit therefore reports the per-class question-length distributions of the benchmark and, within each ground-truth class, compares the lengths of correctly and incorrectly classified items - a within-class length effect would indicate that length, not uncertainty geometry, is

Table 4.3: The seven-criteria filter applied to both source datasets. The criteria are applied a priori as a construct-validity measure, motivated by the shortcut-solvability finding of Min et al. rather than by observed results.

| # | Criterion                                 | Purpose                                                                                                     |
|---|-------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 1 | Question type = bridge                    | Keeps genuine multi-hop chaining; drops comparison questions solvable as two independent single-hop lookups |
| 2 | Difficulty level $\in$ medium, hard       | Drops the easy tier, which clusters with the shortcut-solvable population identified [53]                   |
| 3 | $\geq 2$ distinct supporting-fact titles  | Structural verification that the item genuinely spans multiple documents                                    |
| 4 | Answer $\notin$ yes, no                   | Removes binary answers whose two-value output space produces artificially clean cluster patterns            |
| 5 | Answer length 1-6 words                   | Avoids clustering-sensitivity artifacts on long answers                                                     |
| 6 | Answer not quoted in the question         | Removes answer-leakage annotation errors                                                                    |
| 7 | Question length 7-20 words (both classes) | Length-matching filter applied to HotpotQA and AmbigNQ, reducing the length confound between the classes    |

leaking into the decision. The audit results are reported in Section 5.7 and the residual difference is disclosed as a construct-validity limitation.

#### 4.3.5 Baseline systems

Three baselines span the policy space along the dimension the thesis isolates, all sharing the same base model, the same sampling and clustering machinery, the same confidence estimator and threshold, the same tool budget rules, and the same data which are represented in Table 4.4.

Because B2 and B3 use the identical confidence estimator and threshold as UTSR’s certainty bar, any measured difference between them and UTSR is attributable to the type signal and the typed routes - the intended experimental isolation is enforced in code rather than merely asserted.

#### 4.3.6 Metrics

The dual failure-mode framework measures the two failure modes separately rather than folding them into a single score.

**Classification quality:** per-class precision, recall, and F1 for the epistemic and aleatoric classes, and their macro average; baselines’ actions are mapped to type predictions by a fixed, pre-registered scheme (answering directly is scored as a certainty-side decision; B2’s unstructured threshold escalation is scored on the escalate-to-human side; B1’s and B3’s forced answers never predict aleatoric). Because none of the baselines possesses an aleatoric route, their aleatoric F1 is zero by construction; the comparison on this metric therefore demonstrates the presence of a capability the baselines lack, and the more informative baseline contrasts lie in the failure-mode counts. Certain-control outcomes are reported separately as raw proportions with the corresponding per-class scores.

**FM1 metrics:** wasted escalations (ground-truth epistemic items escalated to a human) and, as the complementary safety metric, missed escalations (ground-truth aleatoric items answered with a

Table 4.4: The three baseline systems. Each isolates one prevailing approach: B1 the no-escalation agent, B2 the magnitude-threshold router that dominates industrial practice, and B3 the type-blind agentic reflection loop. All four systems share the same sampler, clusterer and confidence estimator, so differences are attributable to the routing policy alone.

| System                    | Policy                                                                                                                                                                                            | Expected failure mode                                                                                                      |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| B1 - No Escalation        | Samples, clusters, and always returns the majority-cluster answer; never escalates, never calls tools                                                                                             | Silent forced answers on every ambiguous item (all 26 aleatoric items missed by construction).                             |
| B2 - Fixed Threshold      | Answers if the largest-cluster confidence clears 0.65; otherwise escalates with an unstructured “confidence below threshold” message; type-blind                                                  | FM1: epistemic items below threshold are escalated even though retrieval would have resolved them; no aleatoric detection. |
| B3 - AUQ-style Reflection | While confidence is below 0.65 and the three-call budget permits: call the tool, inject the result, re-sample all $N$ , re-check; type-blind reflection in the style of agentic-UQ loops [36, 38] | FM2: aleatoric items consume the budget in unresolvable reflection before a forced answer.                                 |

forced single answer instead of being escalated - the dangerous error, since the user receives an arbitrary reading of an ambiguous question presented as fact).

**FM2 metrics:** wasted tool calls on aleatoric items and the corresponding waste rate (share of all accounted calls spent on aleatoric items). One accounting convention matters for interpretation: UTSR counts its initial sampling round as one call per question, so every aleatoric item registers exactly one accounted call even when zero retrieval is executed; the baselines’ counters register retrieval calls only. The waste comparison is therefore conservative against UTSR.

**Efficiency:** mean wall-clock seconds per question and total run time per system.

#### 4.3.7 Configuration and protocol

The final configuration across all the models and both dataset conditions are detailed in Table 4.5.

The protocol runs every question through all four systems under identical conditions, resetting pipeline memory and tool counters per question, and serialises per-system results (predicted type, routing agent, escalation flag, tool calls, wall-clock time) to JSON, from which all metrics, tables, and figures are computed by a standalone analysis stage. The full campaign runs the filtered 67-item benchmark (52 powered items plus 15 certain controls) on all three models with all four systems - each executed on all three models with all four systems: twelve system-runs in total. The filtered, three-model condition is the main result.

Table 4.5: Final configuration, held fixed across all models and both dataset conditions.

| Parameter                                        | Value              | Role                                                             |
|--------------------------------------------------|--------------------|------------------------------------------------------------------|
| N_SAMPLES / TEMPERATURE                          | 8 / 0.60           | Samples per routing decision; sampling temperature.              |
| CONFIDENCE_THRESHOLD                             | 0.65               | Certainty bar (R1), early-stop bar, and B2/B3 threshold.         |
| Low / High entropy thresholds                    | 2.00 / 2.50        | Aleatoric bounded-entropy limit; epistemic scatter floor.        |
| POLARISE_THRESHOLD                               | 0.60               | Top-two dominance for the primary aleatoric signal.              |
| MIN_CLUSTERS_EPISTEMIC                           | 5                  | Cluster-count floor for the scatter rule.                        |
| DELTA_THRESHOLD                                  | 0.10               | Minimum consistency gain for the context-injection test.         |
| Similarity / Entailment / Contradiction          | 0.70 / 0.55 / 0.35 | Three-condition cluster-merge gate.                              |
| N_EPISTEMIC / N_ALEATORIC / Certain controls     | 26 / 26 / 15       | Benchmark composition (Power basis for the 26-item groups [52]). |
| MAX_RETRIEVAL_ATTEMPTS / MAX_TOOL_CALLS_BASELINE | 2 / 3              | UTSR retrieval budget; B3 reflection budget.                     |
| RANDOM_SEED                                      | 42                 | Deterministic dataset sampling and shuffling.                    |
| MAX_TOKENS                                       | 150                | Maximum generation length per sample                             |

# 5 Experimental Evaluation and Results

This chapter reports the results of the full experimental campaign and discusses them. Section 5.1 gives the headline comparison of UTSR against the three baselines on all three models under the filtered benchmark. Sections 5.2-5.4 decompose the headline into per-class classification quality, failure-mode outcomes, and routing behaviour. Section 5.5 presents the cross-model analysis, Section 5.6 the effect of the dataset filter, Section 5.7 the length-confound audit, and Section 5.8 the efficiency results. Throughout, “filtered” refers to the seven-criteria benchmark of Section 4.3.3 (26 epistemic + 26 aleatoric powered items, plus 15 certain controls run alongside); F1 metrics are computed over the powered epistemic/aleatoric items, and certain-control outcomes are reported separately.

## 5.1 Headline Comparison

Table 5.1 consolidates the primary metrics for all four systems on all three models under the filtered benchmark.

| Model          | System      | Macro F1 (E+A) | Macro F1 (3 cls) | Certain F1 | Epistemic F1 | Aleatoric F1 | Wasted esc. | Missed esc. |
|----------------|-------------|----------------|------------------|------------|--------------|--------------|-------------|-------------|
| Llama 3.2 (3B) | <b>UTSR</b> | <b>0.389</b>   | <b>0.314</b>     | 0.162      | 0.167        | <b>0.612</b> | <b>10</b>   | <b>5</b>    |
| Llama 3.2 (3B) | B1          | 0.000          | 0.122            | 0.366      | 0.000        | 0.000        | 0           | 26          |
| Llama 3.2 (3B) | B2          | 0.194          | 0.167            | 0.111      | 0.389        | 0.000        | 14          | 7           |
| Llama 3.2 (3B) | B3          | 0.000          | 0.122            | 0.366      | 0.000        | 0.000        | 8           | 16          |
| Llama 3.1 (8B) | <b>UTSR</b> | <b>0.432</b>   | <b>0.320</b>     | 0.095      | 0.375        | <b>0.490</b> | <b>13</b>   | <b>2</b>    |
| Llama 3.1 (8B) | B1          | 0.000          | 0.122            | 0.366      | 0.000        | 0.000        | 0           | 26          |
| Llama 3.1 (8B) | B2          | 0.259          | 0.222            | 0.148      | 0.519        | 0.000        | 21          | 5           |
| Llama 3.1 (8B) | B3          | 0.000          | 0.122            | 0.366      | 0.000        | 0.000        | 13          | 7           |
| GPT-4o-mini    | <b>UTSR</b> | <b>0.376</b>   | <b>0.251</b>     | 0.000      | 0.475        | <b>0.278</b> | <b>20</b>   | <b>1</b>    |
| GPT-4o-mini    | B1          | 0.000          | 0.122            | 0.366      | 0.000        | 0.000        | 0           | 26          |
| GPT-4o-mini    | B2          | 0.256          | 0.171            | 0.000      | 0.512        | 0.000        | 22          | 3           |
| GPT-4o-mini    | B3          | 0.000          | 0.122            | 0.366      | 0.000        | 0.000        | 21          | 5           |

Table 5.1: Headline results, filtered benchmark. Macro F1 (E+A) averages the epistemic and aleatoric classes and is the primary metric; the three-class variant additionally averages the certain-control class and is reported for completeness. UTSR attains the highest value on both variants on every model. Waste rate is the share of accounted tool calls spent on aleatoric items; UTSR’s accounting includes its sampling rounds (Section 4.3.6), so the comparison is conservative against it.

Three patterns hold on every model. First, UTSR achieves the best macro F1 (0.376-0.432 versus at most 0.259 for the best baseline). Second, UTSR is the only self-answering system with non-zero aleatoric F1 - B1 and B3 never escalate or never type their escalations, and B2’s threshold escalation is type-blind, so all three sit at zero by construction. Third, UTSR has the fewest missed escalations - the dangerous failure in which an ambiguous question receives a forced single answer - with 5, 2, and 1 misses against B1’s structural 26, B2’s 7/5/3, and B3’s 16/7/5. UTSR also issues fewer wasted escalations than the fixed-threshold baseline B2 on every model (10 vs 14, 13 vs 21, 20 vs 22), while B1’s zero wasted escalations is degenerate (it never escalates anything, at the price of missing all 26 aleatoric items) and B3 achieves its counts only by forcing answers on most timed-out items. Figure 5.1 shows the cross-model view and Figure 5.2 the corresponding comparison against all three baselines. Figures 5.3 to 5.5 present the per-model failure-mode detail and Figure 5.6 the routing distribution for Llama 3.1 (8B), chosen as the illustrative case because it sits at the mid-point of the capability range and exhibits neither the 3B

model’s false-certainty failure nor GPT-4o-mini’s collapsed aleatoric fingerprint in isolation. It is also the model on which UTSR performs best (Table 5.1).

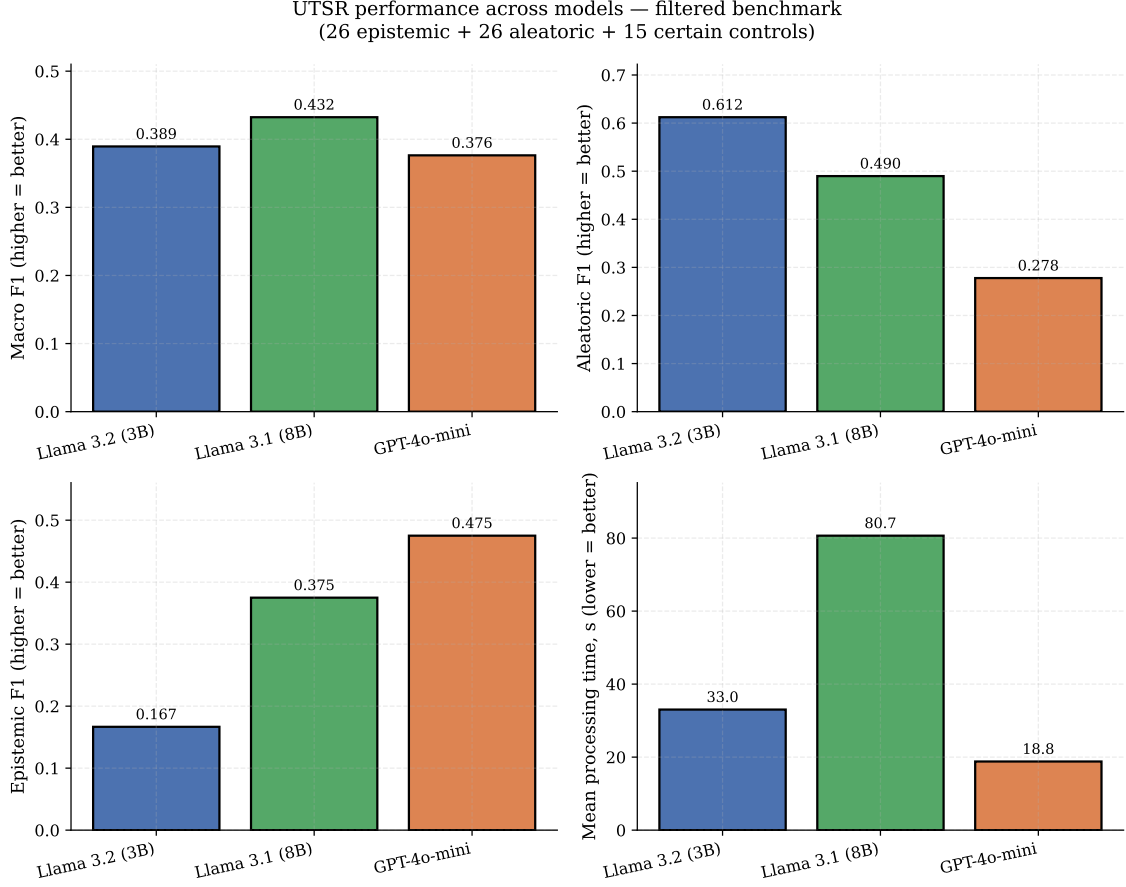


Figure 5.1: UTSR on the filtered benchmark across the three evaluated models (26 epistemic, 26 aleatoric, 15 certain controls). Aleatoric F1 falls monotonically with model capability (0.612  $\rightarrow$  0.490  $\rightarrow$  0.278) while epistemic F1 rises (0.167  $\rightarrow$  0.375  $\rightarrow$  0.475); this mirror-image pattern is the central cross-model finding and is analysed in Section 5.5.

## 5.2 Classification Quality by Class

Table 5.2 reports UTSR’s per-class precision, recall, and F1 on the filtered benchmark, including the certain-control class.

Two observations stand out. First, epistemic and aleatoric performance move in opposite directions across models: aleatoric F1 falls monotonically with model capability (0.61  $\rightarrow$  0.49  $\rightarrow$  0.28) while epistemic F1 rises (0.17  $\rightarrow$  0.375  $\rightarrow$  0.475). This mirror-image pattern is the central cross-model finding and is analysed in Section 5.5. Second, the certain-control class fails on all three models: only 3/15 (3B), 1/15 (8B), and 0/15 (GPT-4o-mini) trivial questions were routed to the direct-answer path (exact binomial 95% confidence intervals roughly 0.07-0.45, 0.01-0.30, and 0.00-0.20 respectively). Notably, this false-uncertainty failure is shared by the baselines that use the same confidence estimator: B2 escalated 13 of the 15 certain controls on the 8B model, essentially the same proportion as UTSR’s 14. The failure therefore lies upstream of the routing rules - in the confidence estimate produced by the strict semantic clusterer - and is diagnosed in Section 6.4.

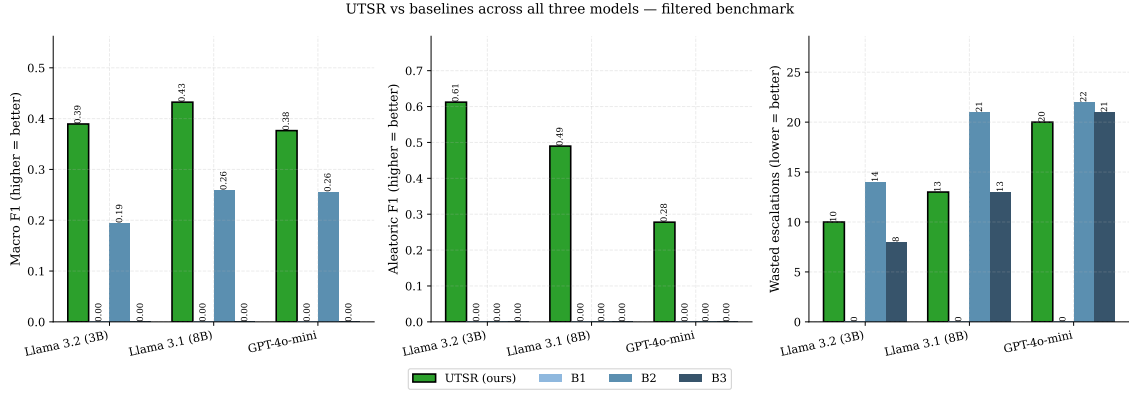


Figure 5.2: UTSR against the three baselines on all three models, filtered benchmark. UTSR attains the highest macro F1 on every model and is the only system with non-zero aleatoric detection. B1’s zero wasted escalations are degenerate: it never escalates and consequently misses all 26 ambiguous items on every model.

| Model          | Class             | Precision | Recall | F1    |
|----------------|-------------------|-----------|--------|-------|
| Llama 3.2 (3B) | Certain (control) | 0.136     | 0.200  | 0.162 |
|                | Epistemic         | 0.182     | 0.154  | 0.167 |
|                | Aleatoric         | 0.652     | 0.577  | 0.612 |
| Llama 3.1 (8B) | Certain (control) | 0.167     | 0.067  | 0.095 |
|                | Epistemic         | 0.316     | 0.462  | 0.375 |
|                | Aleatoric         | 0.522     | 0.462  | 0.490 |
| GPT-4o-mini    | Certain (control) | 0.000     | 0.000  | 0.000 |
|                | Epistemic         | 0.352     | 0.731  | 0.475 |
|                | Aleatoric         | 0.500     | 0.192  | 0.278 |

Table 5.2: UTSR per-class precision, recall, and F1 scores for each evaluated model on the filtered benchmark. Certain-control metrics are computed over the 15-item sanity set, while epistemic and aleatoric metrics are computed over the corresponding 26-item subsets.

Comparing against the baselines per class (Table 5.1): B2 attains the best epistemic F1 on every model (0.389 / 0.519 / 0.512 versus UTSR’s 0.167 / 0.375 / 0.475). This is not evidence that the threshold baseline understands epistemic uncertainty better; it is the arithmetic of blanket escalation. B2 labels every below-threshold item as escalation-worthy, which on this benchmark yields epistemic recall of 0.54-0.84 at the cost of also sweeping up the aleatoric items (whence its 14-22 wasted escalations and zero aleatoric F1). UTSR deliberately splits the below-threshold mass between two routes, trading some epistemic recall for the ability to detect aleatoric uncertainty at all. Section 6.2 argues that single-class F1 is the wrong lens for this trade and that the failure-mode counts are the decision-relevant metrics.

### 5.3 Failure-Mode Results

FM1 - escalation behaviour. The decision-relevant pairing is wasted escalations (unnecessary human load) against missed escalations (dangerous forced answers), since either can be trivially optimised at the other’s expense: B1 achieves zero waste by never escalating and consequently misses all 26 ambiguous items on every model; B3 forces answers on most timed-out items with

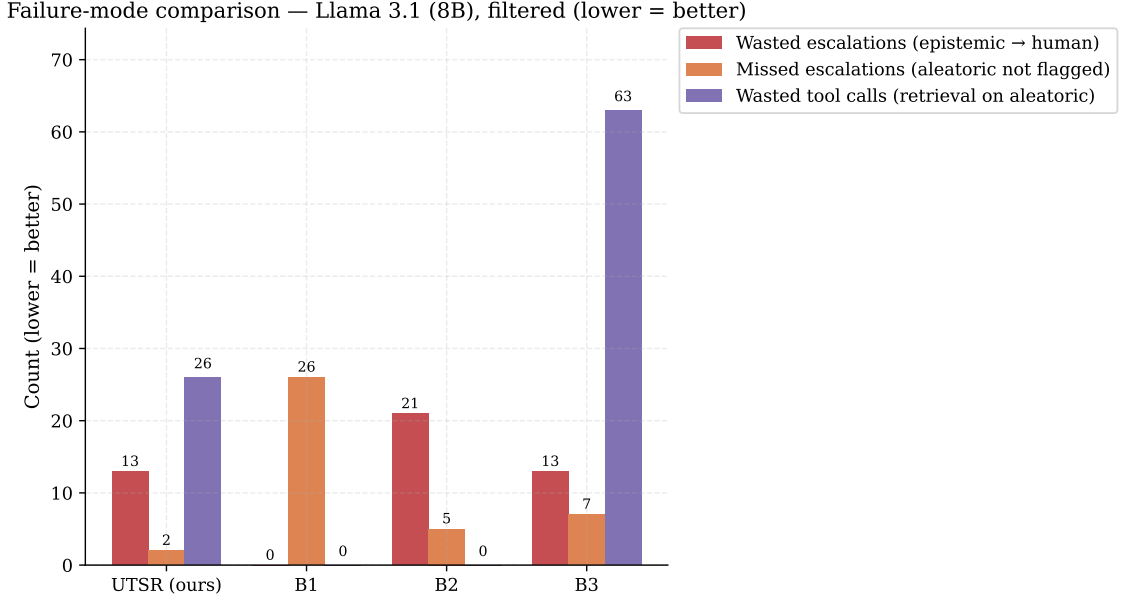


Figure 5.3: The three failure-mode counts on Llama 3.1 (8B), filtered benchmark; lower is better on all three. B3 exhibits the pathology it was constructed to represent, expending 63 futile tool calls on aleatoric items without resolving one. UTSR’s 26 accounted calls are the initial sampling rounds (Section 4.3.6); its aleatoric route executes zero retrieval by construction.

the same effect (16/7/5 misses). Against the only serious competitor, B2, UTSR strictly dominates on both counts on every model: fewer wasted escalations ( $10 < 14$ ,  $13 < 21$ ,  $20 < 22$ ) and fewer missed escalations ( $5 < 7$ ,  $2 < 5$ ,  $1 < 3$ ). Beyond the counts, the content of the escalations differs qualitatively: every UTSR escalation - clarification or human handoff - carries enumerated, support-weighted candidate interpretations and, for the retrieval branch, the full audit trail of attempts and  $\Delta$  values, whereas B2’s escalation is a bare “confidence below threshold” flag. On the 8B model, 24 of 26 aleatoric items were escalated in structured form even though only 12 carried the correct aleatoric label (the other 12 travelled via the retrieval branch, whose empty simulated corpus for aleatoric items terminates the loop after a single empty retrieval and hands off with candidates attached). At the level of the action taken - surface the ambiguity to a human with options rather than guess - UTSR’s aleatoric escalation recall is thus 92% on the 8B model, materially higher than its label-level aleatoric recall of 46%. The distinction between label-level and action-level performance recurs in the discussion.

FM2 - tool-call waste. B3 exhibits exactly the failure it was constructed to represent: on aleatoric items its reflection loop retrieves, re-samples, and re-checks until the budget exhausts, accumulating 48 (3B), 63 (8B), and 67 (GPT-4o-mini) futile tool calls - a 42% / 42% / 37% waste rate - without a single aleatoric item resolving, precisely the unresolvable-reflection pathology anticipated for type-blind self-correction [38]. UTSR’s aleatoric route executes zero retrieval calls by construction; its recorded 26 “wasted” calls per model are the accounting convention of Section 4.3.6 (one initial sampling round per aleatoric item), and even under this conservative accounting its waste rate (25-35%) sits below B3’s on every model. One honest caveat: aleatoric items misrouted into the epistemic branch do attempt retrieval; in this benchmark the attempt terminates immediately because the aleatoric corpus is empty, but with a real retriever each such misroute

Failure-mode comparison — Llama 3.2 (3B), filtered (lower = better)

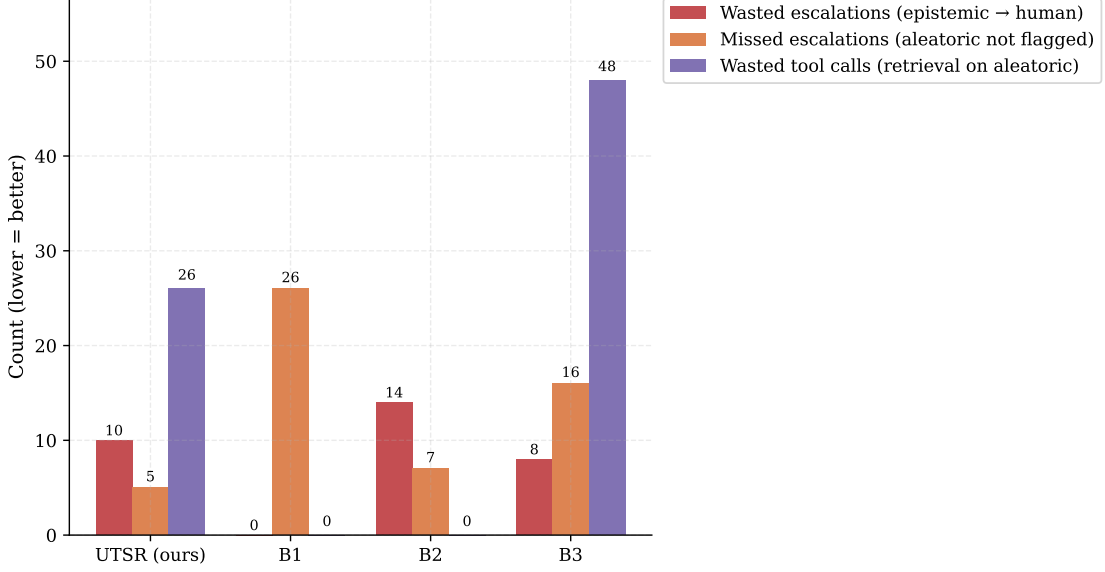


Figure 5.4: Failure-mode counts on Llama 3.2 (3B), filtered benchmark; lower is better. The ordering matches the mid-scale case of Figure 5.3: UTSR issues fewer wasted and fewer missed escalations than B2 (10 against 14, and 5 against 7), while B3 expends 48 futile tool calls.

would cost up to the two-attempt budget. The structural FM2 guarantee therefore holds unconditionally only for correctly classified items, and degrades gracefully - bounded by the attempt budget - for misclassified ones.

The baselines are chosen to span the policy space along the dimension the thesis isolates. B1 establishes the floor: what happens with no uncertainty handling at all. B2 represents the dominant industrial pattern - scalar confidence with threshold escalation - and the fixed-DAG escalation designs of Section 3.4. B3 represents the reflection-centric research frontier, which handles epistemic uncertainty well but is type-blind. UTSR differs from B2 and B3 only in possessing the type signal and typed routes; any measured difference in the failure-mode metrics is therefore attributable to the type mechanism, which is the intended experimental isolation.

## 5.4 Routing Behaviour

Table 5.3 summarises where each ground-truth class ended up under UTSR (percentages of the class; Figure 5.6 shows the full per-system distribution for the 8B model).

The per-item traces of the 8B run make the epistemic branch’s internal behaviour precise: of the twelve epistemic items that entered the retrieval loop, the context-injection delta test resolved ten - the injected gold paragraphs collapsed the answer distribution and the converged answer was emitted autonomously - while only two exhausted the budget and reached human handoff. The same derivation for GPT-4o-mini yields roughly four resolutions out of nineteen entries (about one in five), and for the 3B model roughly two of four. The delta test therefore works as designed when the geometry upstream of it is healthy (8B), and its failure on GPT-4o-mini is inherited from a distorted input distribution rather than from the test’s logic - the diagnosis in Section 6.3. The routing table also localises each model’s characteristic error: the 3B model’s dominant failure

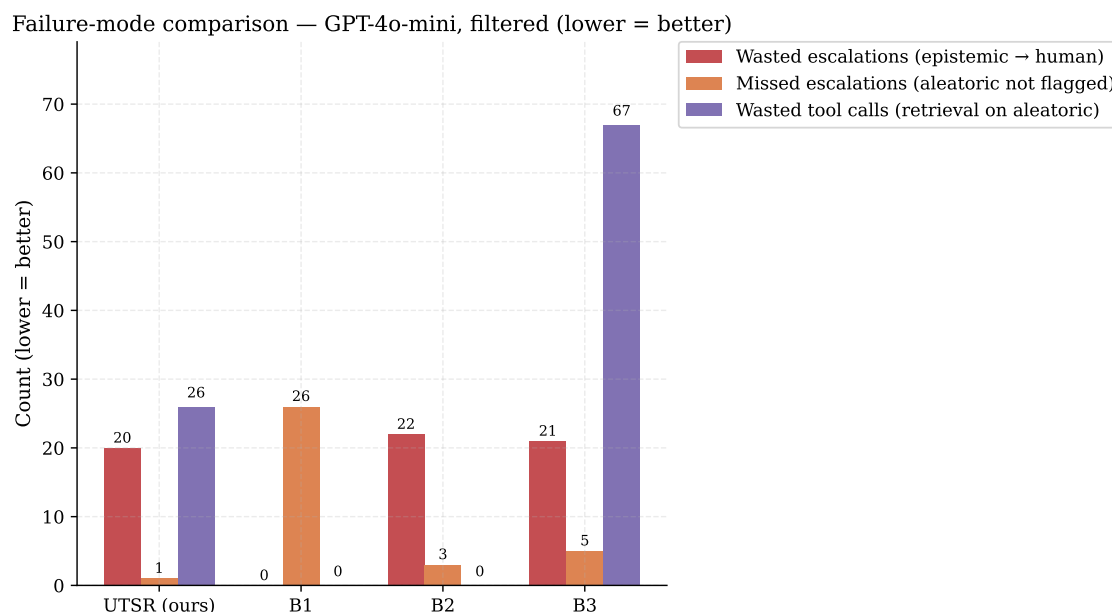


Figure 5.5: Failure-mode counts on GPT-4o-mini, filtered benchmark; lower is better. UTSR again issues fewer wasted and fewer missed escalations than B2 (20 against 22, and 1 against 3). B3’s 67 futile tool calls are the campaign’s highest, reflecting the reflection loop’s inability to terminate on ambiguous items.

is false certainty on epistemic items (54% answered directly - consistent, confidently repeated answers to multi-hop questions it cannot verify); GPT-4o-mini’s dominant failure is the disappearance of the aleatoric fingerprint (77% of ambiguous items looked like knowledge gaps); the 8B model sits between the two extremes and, correspondingly, achieves the best macro F1.

## 5.5 Cross-Model Analysis: the Signal Depends on the Base Model’s Sampling Entropy

The mirror-image pattern of Table 5.2 - aleatoric F1 falling  $0.61 \rightarrow 0.49 \rightarrow 0.28$  while epistemic F1 rises  $0.17 \rightarrow 0.375 \rightarrow 0.475$  as model capability increases - is the most consequential finding of the evaluation, and it admits a coherent single explanation: the cluster-geometry signal is a function of the base model’s sampling entropy, and the fixed thresholds were calibrated at one point on that spectrum.

At the low-capability end, Llama 3.2 (3B) produces high-variance samples. This helps aleatoric detection - genuinely ambiguous questions readily display their multiple readings as separate camps, yielding the campaign’s best aleatoric F1 - but it coexists with a failure in the opposite direction on epistemic items: the small model frequently repeats the same unverified answer across all eight samples, producing a single dominant cluster that clears the 0.65 certainty bar. Fifty-four percent of epistemic items were answered directly as false CERTAIN. This is consistent hallucination defeating a consistency-based signal: sampling-agreement methods assume that fabricated content varies across samples [5, 21], and a small model that has settled on one wrong guess violates that assumption. Semantic self-consistency measures conviction, not correctness.

At the high-capability end, GPT-4o-mini exhibits the reverse pathology. At temperature 0.60

Table 5.3: Distribution of UTSR routing decisions by model and ground-truth (GT) class on the filtered benchmark. Values indicate the percentage of questions routed directly to an answer, through the epistemic retrieval/escalation path, or through the aleatoric clarification path.

| Model          | GT class  | Answered directly | Epistemic path      | Aleatoric path |
|----------------|-----------|-------------------|---------------------|----------------|
|                |           |                   | (retrieve/escalate) | (clarify)      |
| Llama 3.2 (3B) | Aleatoric | 19%               | 23%                 | 58%            |
|                | Certain   | 20%               | 80%                 | 0%             |
|                | Epistemic | 54%               | 15%                 | 31%            |
| Llama 3.1 (8B) | Aleatoric | 8%                | 46%                 | 46%            |
|                | Certain   | 7%                | 93%                 | 0%             |
|                | Epistemic | 11%               | 46%                 | 42%            |
| GPT-4o-mini    | Aleatoric | 4%                | 77%                 | 19%            |
|                | Certain   | 0%                | 100%                | 0%             |
|                | Epistemic | 8%                | 73%                 | 19%            |

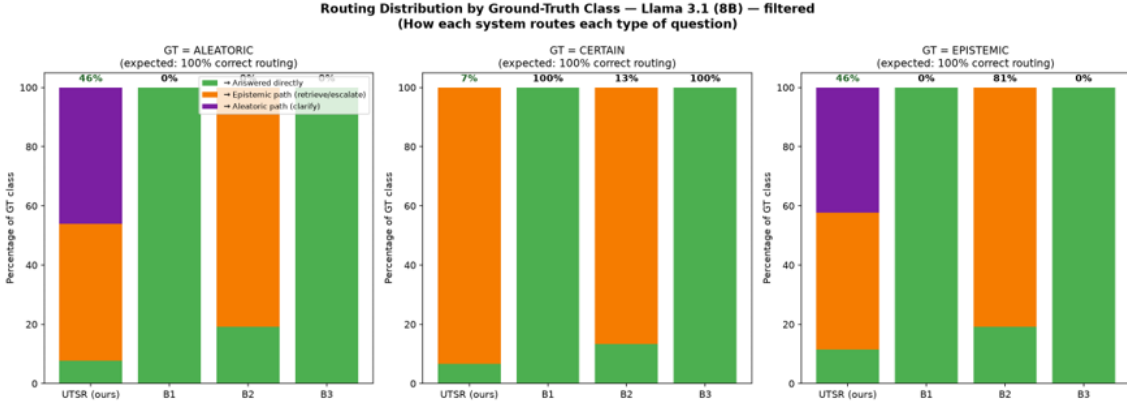


Figure 5.6: Routing distribution by ground-truth class on Llama 3.1 (8B), all systems, filtered benchmark. The certain controls fail conspicuously: 93% reach the human-handoff agent, a failure localised to the shared confidence estimator in Section 6.4 rather than to the routing rules.

a strong instruction-tuned model is highly consistent: on many ambiguous questions it decisively commits to one reading (or distributes its remaining variance as paraphrase-level rather than interpretation-level differences), so the polarised two-camp fingerprint that the aleatoric rules key on collapses or fragments, and 77% of ambiguous items presented as scattered or unconverged geometries that routed into the epistemic branch. This is a known scaling behaviour of sampling-based uncertainty signals - as base-model consistency improves, the distributional signal available to self-consistency and semantic-entropy methods weakens [6,21] - here observed specifically for the type signal rather than the magnitude signal. The mid-scale Llama 3.1 (8B) occupies the sweet spot: enough sampling entropy to expose ambiguity (aleatoric F1 0.49), enough capability to avoid mass false certainty (epistemic F1 0.375), the healthiest delta-test behaviour (ten of twelve retrieval entries resolved), and consequently the best macro F1 of the campaign (0.432).

Two things follow. Negatively: the mechanism as configured is not scale-invariant - a single threshold set calibrated on a 3B model does not transfer unchanged across the capability range

from a small local open-weight model to a proprietary cloud model, and the claim of model-agnosticism must be weakened from “identical configuration works everywhere” to “the mechanism works everywhere given per-model calibration.” Positively: the direction of every headline comparison is nevertheless preserved on all three models - UTSR beats every baseline on macro F1 and aleatoric F1, dominates B2 on both escalation counts, and beats B3 on waste and latency at every scale - which is the classical robust-in-trend argument for small-n, multi-model evidence. The finding also has an actionable signature: the base model’s raw sampling entropy on a small probe set predicts which failure direction to expect, suggesting a cheap per-model calibration procedure (Section 7.5).

## 5.6 Effect of the Dataset Filter

Table 5.4 compares UTSR under the unfiltered against the filtered condition.

Table 5.4: UTSR before and after the seven-criteria filter.

| Model          | Macro F1   |          | Aleatoric F1 |          |
|----------------|------------|----------|--------------|----------|
|                | Unfiltered | Filtered | Unfiltered   | Filtered |
| Llama 3.2 (3B) | 0.610      | 0.389    | 0.513        | 0.612    |
| Llama 3.1 (8B) | 0.409      | 0.432    | 0.318        | 0.490    |
| GPT-4o-mini    | 0.345      | 0.376    | 0.118        | 0.278    |

The filter’s effect is asymmetric and informative. For the two larger models it improved both macro and aleatoric F1 - removing yes/no answers and length outliers gave the clusterer cleaner answer spaces, and aleatoric F1 rose substantially ( $0.318 \rightarrow 0.490$  and  $0.118 \rightarrow 0.278$ ). For the 3B model, macro F1 dropped sharply ( $0.610 \rightarrow 0.389$ ), driven by the epistemic side: the unfiltered epistemic class contained comparison-type, easy-tier, and binary-answer questions - exactly the shortcut-solvable population documented by [53] - on which even a 3B model produces clean, classifiable geometries. Filtering to genuinely multi-hop bridge questions made the epistemic class harder and more honest, and the small model’s consistent-hallucination failure (Section 5.5) then dominated. The unfiltered 0.610 should therefore be read as partially inflated by dataset artifacts, and the filtered numbers as the defensible ones; reporting both, with the drop explained, is itself a rigour contribution of the campaign - the filter functioned as intended, removing a source of apparent performance that did not reflect the construct being measured.

## 5.7 Length-Confound Audit

The audit of Section 4.3.4 finds that, even after the 7-20-word length-matching filter, a residual difference remains between the classes: filtered HotpotQA questions average 15.3 words against 9.3 for filtered AmbigNQ (Figure 5.7) - a moderate, disclosed confound of six words; the distributions overlap in the 9-16 word band but each class retains a clearly separated mode, much reduced from the unmatched raw pools but not eliminated, since the two datasets’ natural length ranges only partially overlap. Two considerations bound the threat. First, the classifier never inspects the question text; length could influence classification only indirectly, through the diversity of the sampled answers. Second, the within-class check - comparing question lengths of correctly and

incorrectly classified items inside each ground-truth class - is the diagnostic that would expose such leakage, and it is part of the released analysis tooling; the residual between-class difference is disclosed in the limitations (Section 7.4) with the note that a fully length-matched benchmark would require either truncating HotpotQA’s naturally longer multi-hop phrasings or extending AmbigNQ’s naturally shorter ones, both of which would distort the constructs being measured.

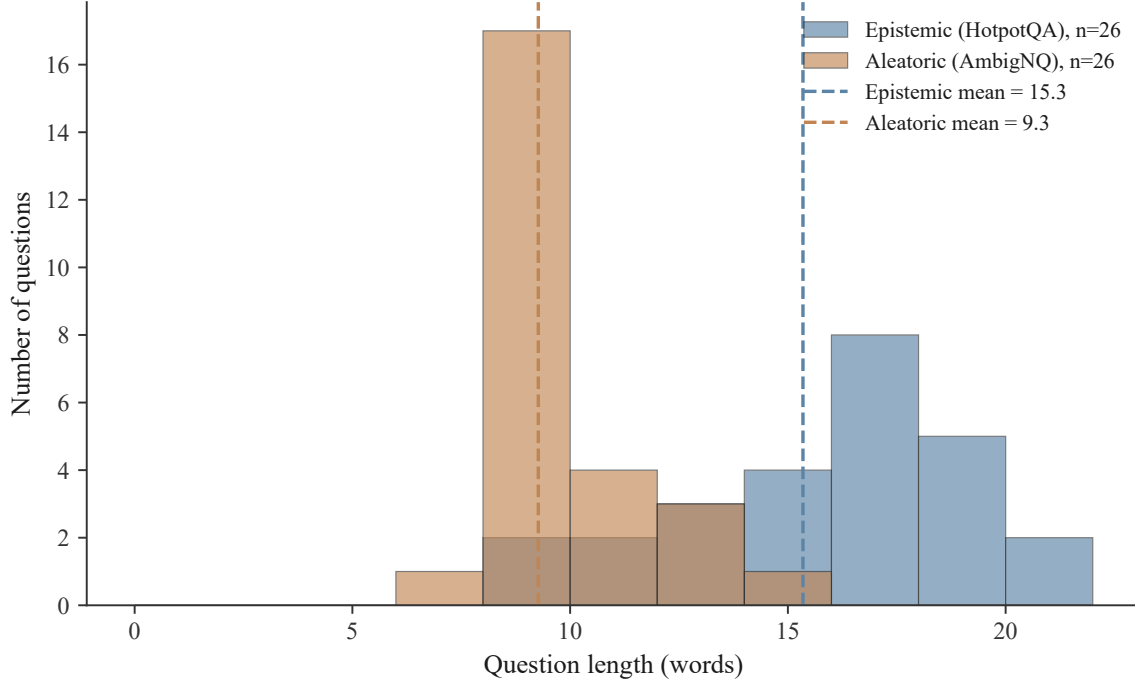


Figure 5.7: Question-length distributions by class on the filtered benchmark. Means: epistemic 15.3 words, aleatoric 9.3 words. The distributions overlap in the 9 to 16 word band but each class retains a clearly separated mode; the residual six-word difference is the disclosed confound discussed in Section 5.7.

## 5.8 Efficiency

UTSR’s latency position is consistent across models: it pays a bounded sampling-and-classification overhead relative to the non-reflective baselines and undercuts the reflection baseline by a wide margin. Mean per-question times (filtered benchmark) are 33.0 s versus B1/B2’s 28 s on the 3B model, 80.7 s versus 51.6 s on the 8B model, and 18.8 s versus 12.6 s on GPT-4o-mini - an overhead factor of 1.2-1.6× - while B3’s type-blind reflection costs 77.3 s, 142.5 s, and 45.7 s respectively. Total wall-clock time for a full four-system filtered run was roughly 186 minutes (3B, local), 364 minutes (8B, local), and 100 minutes (GPT-4o-mini, cloud); the corresponding unfiltered full runs measured 172, 336, and 76 minutes. These runtimes substantiate the sample-size argument of Section 4.3.2: the campaign’s cost is dominated by LLM sampling and pairwise NLI scoring, and scaling the powered classes by an order of magnitude is a hardware budget question, not a design question. Mean seconds per question and total wall-clock minutes on Llama 3.2 (3B), Llama 3.1 (8B) and GPT-4o-mini are represented in Figures 5.8, 5.9 and 5.10 respectively.

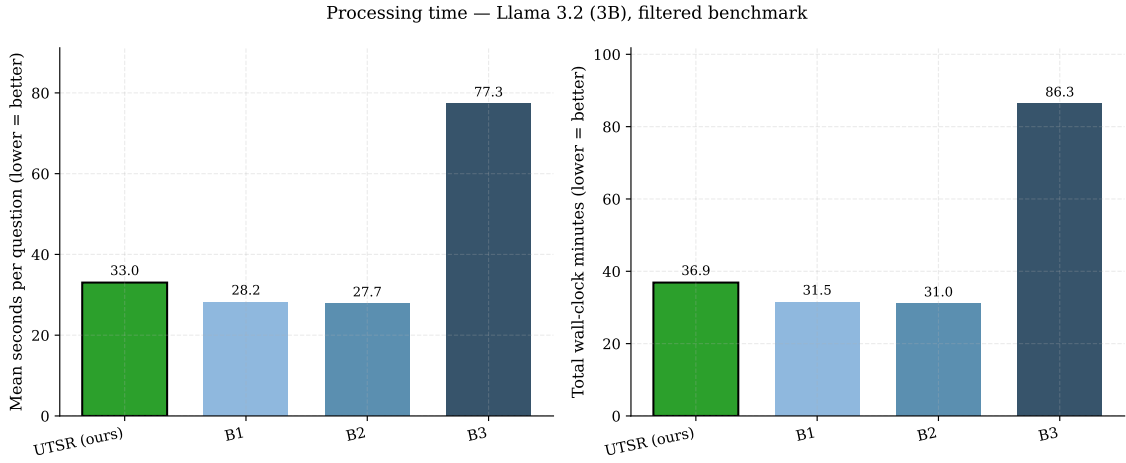


Figure 5.8: Mean seconds per question and total wall-clock minutes on Llama 3.2 (3B), filtered benchmark. UTSR’s overhead over the non-reflective baselines is 1.2×; it completes 2.3× faster than B3.

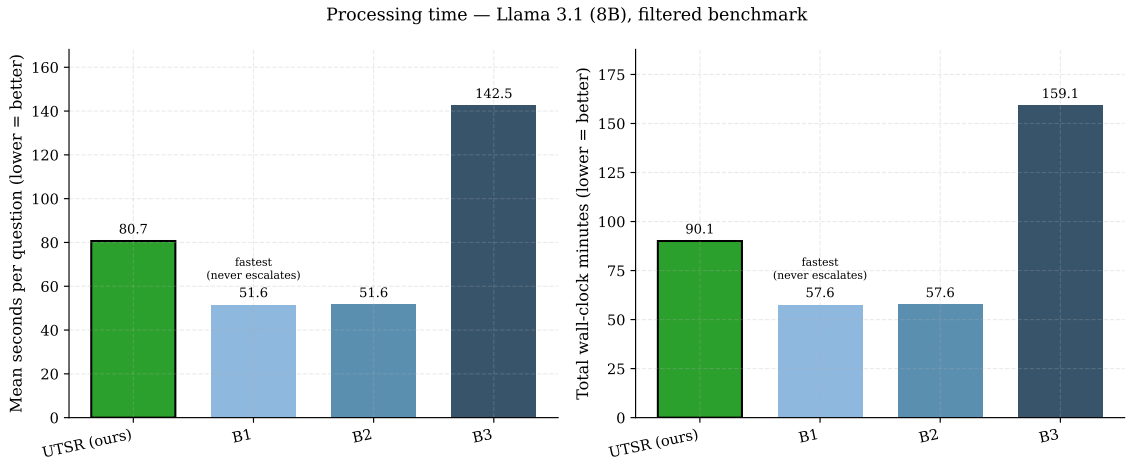


Figure 5.9: Mean seconds per question and total wall-clock minutes on Llama 3.1 (8B), filtered benchmark. UTSR pays a bounded overhead of 1.6× over the non-reflective baselines and completes 1.8× faster than the reflection baseline B3. B1’s advantage is degenerate: it never escalates and never retrieves.

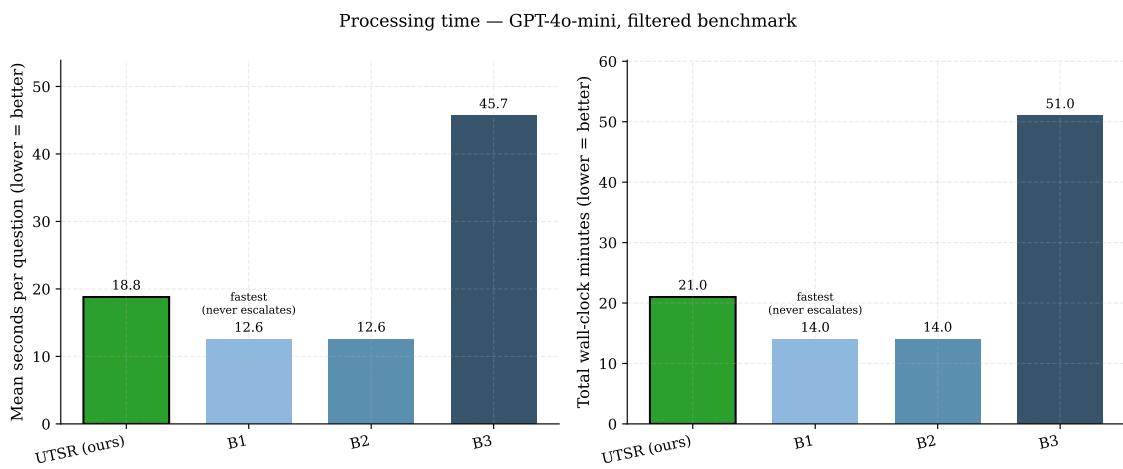


Figure 5.10: Mean seconds per question and total wall-clock minutes on GPT-4o-mini, filtered benchmark. UTSR’s overhead over the non-reflective baselines is 1.5×; it completes 2.4× faster than B3.

## 6 Discussion

The findings of Chapter 5 against the hypotheses are discussed in this chapter. The preceding sections reported what the systems did; this one asks what those numbers support. The results are mixed in an informative way: the structural claims of the thesis hold cleanly, while the statistical ones hold in direction but not in magnitude, and one hypothesis fails in a manner that turns out to be the more interesting finding. The discussion therefore proceeds from the hypotheses to the individual mechanisms whose behaviour explains them - the delta test, the clusterer, and the escalation metric itself - before returning to the question of how far the evidence licenses the thesis's central claim.

### 6.1 The hypotheses revisited

**H1 (type separation from cluster geometry) - partially supported.** UTSR separates the types better than every magnitude-only baseline on every model, and it is the only self-answering system with any aleatoric detection at all. However, the absolute F1 levels are modest (macro 0.376-0.432 filtered), far from the near-clean separation the fingerprint hypothesis idealises, and decisively the separation quality is strongly model-dependent. The fingerprints exist, but their sharpness is a property of the (model, temperature, threshold) triple, not of the questions alone.

**H2 (fewer wasted escalations than fixed-threshold routing at matched or better safety) - supported in direction on all three models.** UTSR issues fewer wasted escalations than B2 everywhere (29%, 38%, and 9% fewer on 3B, 8B, and GPT-4o-mini respectively) while simultaneously missing fewer ambiguous items - strict dominance on the FM1 pairing. The margins on GPT-4o-mini are thin, and at  $n = 26$  per class none of the pairwise escalation-count differences should be presented as statistically significant; the claim is a consistent direction across three heterogeneous models, which is the appropriate strength of claim for this sample size.

**H3 (elimination of unresolvable reflection on aleatoric items) - supported.** The aleatoric route executes zero retrieval by construction; B3 burned 48-67 futile calls per model at 37-42% waste without resolving a single ambiguous item; and UTSR completes 1.8-2.4 $\times$  faster than B3 on every model. This is the thesis's cleanest result, and it is structural rather than statistical.

**H4 (cross-model robustness) - partially supported, with the failure itself constituting the finding.** The headline orderings are preserved at every scale (robust in trend), but the per-class quality shifts dramatically and in opposite directions with model capability. The fixed configuration is not scale-invariant; the mechanism plus per-model calibration plausibly is, and demonstrating that is the first item of future work.

### 6.2 Why the baselines win individual classes - and why those are the wrong metrics

B2's epistemic F1 advantage (up to 0.519) invites the question of whether the added machinery of typed routing is worth it. The answer lies in what B2's score is made of: it escalates every below-threshold item, achieving epistemic recall of 0.54-0.84 by over-triggering the human, which is precisely the behaviour type-aware routing exists to avoid. The learning-to-defer literature makes

the underlying point formally - deferral policy quality cannot be read off a single class's F1 but must reflect the comparative cost of machine and human effort [41] - and the failure-mode counts operationalise exactly that: on the pairing that matters (wasted escalations and missed escalations, both lower-is-better), UTSR strictly dominates B2 on all three models. A system that pushed epistemic F1 by escalating more would move along B2's frontier, not past it.

### 6.3 The delta test: sound logic, distribution-dependent input

The context-injection delta test behaved exactly as designed wherever its input geometry was healthy: on the 8B model, ten of the twelve epistemic items entering the retrieval loop were resolved autonomously by injecting the gold paragraphs, with only two reaching human handoff. Its apparent failure on GPT-4o-mini (roughly one resolution in five entries, and 20 of 26 epistemic items ultimately escalated) traces upstream, to the same distributional distortion documented in Sections 5.5 and 6.4: when the post-injection samples fragment into paraphrase-level clusters, the post-injection confidence stays below what the additive gain criterion ( $\Delta \geq 0.10$  measured toward a 0.65 bar) needs, and the loop times out even though the model's answers have converged in meaning. The test's behavioural logic - treat uncertainty as epistemic exactly when the admissible intervention measurably reduces it - survives this evaluation intact; what needs revision is the fragility of its confidence input, and possibly the additive form of the criterion itself (a relative gain, or a post-hoc certainty check decoupled from the pre-injection baseline, would be less exposed to ceiling and floor effects).

### 6.4 False uncertainty on the certain controls: a shared clusterer failure

The certain-control results (3/15, 1/15, 0/15 direct answers) are the campaign's clearest negative finding, and the per-item traces plus the baseline comparison localise its cause precisely. On the 8B model, B2 - which shares nothing with UTSR except the sampling, clustering, and confidence machinery - escalated 13 of the 15 controls, statistically indistinguishable from UTSR's 14. The failure therefore lies in the shared confidence estimator, not in the routing rules: the three-condition merge gate ( $\text{similarity} \geq 0.70 \wedge \text{entailment} \geq 0.55 \wedge \text{contradiction} \leq 0.35$ ), tuned deliberately strict to preserve the two-camp aleatoric fingerprint, over-fragments benign paraphrase variation ("Paris", "The capital of France is Paris.", "It's Paris.") into separate clusters, deflating the largest-cluster confidence below the 0.65 bar even on trivially certain questions. The consequence chain is then mechanical: the controls classify epistemic, enter the retrieval branch, find an empty simulated corpus (controls carry no context), and escalate - 14 of 15 reaching the human-handoff agent on the 8B model. This is the mirror image of the over-merging failure the strict gate was introduced to prevent, and it exposes the central calibration tension of the clusterer: the gate must be loose enough to merge paraphrases of the same claim yet strict enough to separate different claims, and a single static threshold triple cannot serve both regimes across models with different verbosity profiles. The same fragmentation mechanism parsimoniously explains GPT-4o-mini's profile - verbose, varied phrasings defeating the merge gate push distributions toward apparent scatter, inflating the epistemic route (100% of controls, 77% of aleatoric items) and starving both the certain and the aleatoric labels - making paraphrase-tolerant clustering the single highest-leverage repair identified by this evaluation.

## 6.5 Label-level versus action-level performance

A recurring nuance deserves explicit statement: several of UTSR’s label-level errors are action-level near-misses. On the 8B model, 12 aleatoric items were mislabelled epistemic - yet all 12 were still escalated, in structured form with enumerated candidate options, because the retrieval branch’s empty-corpus exit hands off rather than guessing. Action-level escalation recall on ambiguous items was thus 92% against a label-level recall of 46%, and only one to five ambiguous items per model (5, 2 and 1 on the 3B, 8B and GPT-4o-mini models respectively) received the genuinely dangerous outcome of a forced answer. For the deployment concern that motivates FM1 and FM2 - do humans see the right cases, with usable structure, and is compute wasted? - the action-level numbers are the operative ones, and on those UTSR’s design degrades gracefully in both misclassification directions, exactly as intended.

## 6.6 Threats to validity revisited

Statistical precision: With 26 items per powered class, single-model F1 differences of the observed magnitude carry wide confidence intervals; the campaign’s evidentiary strategy is direction-consistency across three heterogeneous models rather than significance within any one, and the certain-control set is reported as a raw proportion with a binomial interval, as befits a sanity check rather than a powered comparison. The per-item decision traces required for paired inference - McNemar’s exact test on the escalation counts and bootstrap intervals on the F1 differences - are serialised by the harness and released with the implementation, so these analyses can be run on the existing data without re-executing the campaign; they are reported as future work in Section 7.5 rather than here. Tuning leakage: Thresholds were calibrated on the 3B model against the same benchmark family; the cross-model runs are clean of tuning by construction (fixed configuration), which converts the leakage threat into the measured calibration-transfer finding. Construct validity: The residual six-word length difference between classes is disclosed; HotpotQA remains an imperfect epistemic proxy even after filtering (the filter reduces, not eliminates, shortcut solvability); and the ground-truth types are dataset-level constructs whose noise penalises rather than flatters the measured F1. Simulation: Retrieval is simulated by gold-context injection (guaranteed recall) and escalation is simulated (issued, not consumed); real retrievers make the delta test conservative, and real reviewers are needed to validate the claimed usability of structured escalations. Accounting: The tool-call convention counts UTSR’s sampling rounds, biasing the waste comparison against it; the reported FM2 advantage is therefore a lower bound.

# 7 Conclusion and Future Work

This chapter closes the argument developed over the preceding five chapters and sets out what follows from it. Section 7.1 recapitulates the thesis as a whole: the scalar-uncertainty problem in multi-agent pipelines, the two failure modes it produces, the UTSR mechanism proposed in response, and the three-model evaluation used to test it. Section 7.2 answers each of the four research questions against the evidence of Chapter 5, stating the qualifications alongside the answers rather than deferring them. Section 7.3 consolidates the five contributions, including the two negative findings that constrain how the mechanism should be built next. Section 7.4 bounds these claims in six respects, from the sample size that determines what can be claimed at all to the robustness axes the final campaign displaced. Section 7.5 converts the observed failures into an agenda ordered by leverage, and Section 7.6 returns to the central claim to assess what the evidence ultimately licenses.

The chapter is written to be readable on its own by someone who has not followed the experimental detail, but the qualifications it records are not decorative: the most consequential finding of this work is one the original design did not anticipate, and it is stated here as plainly as the results it complicates.

## 7.1 Summary of the Thesis

This thesis set out from a simple observation about multi-agent LLM pipelines: they treat uncertainty as a scalar. When an agent’s confidence drops below a threshold, current orchestration frameworks apply one fixed remedy - retry, reflect, retrieve, or escalate - regardless of why the agent is uncertain. Decision theory has long distinguished two fundamentally different kinds of uncertainty [10–12]: epistemic uncertainty, which additional information can reduce, and aleatoric uncertainty, which is inherent to an under-specified task and which no retrieval can resolve. Conflating the two produces the pipeline failure modes this thesis formalised: wasted human escalation on retrievable questions (FM1) and unresolvable reflection loops on ambiguous ones (FM2).

The thesis proposed Uncertainty-Type-Driven Subtask Routing (UTSR), a training-free, black-box mechanism that classifies the type of an agent’s uncertainty from the geometry of its sampled answer distribution - scattered clusters indicating epistemic guessing, polarised camps indicating aleatoric ambiguity, one dominant cluster indicating certainty - and dispatches each case to a type-appropriate specialist: direct answering, delta-tested retrieval, or a structured clarification that enumerates the valid interpretations and issues zero retrieval calls. The system was implemented end-to-end on open tooling, evaluated against three baselines spanning the current policy space (no escalation, fixed-threshold escalation, and type-blind AUQ-style reflection), on a benchmark whose ground-truth uncertainty types are inherent to dataset construction - HotpotQA multi-hop questions as the epistemic class [43] and multi-answer AmbigNQ questions as the aleatoric class [44] - hardened by a seven-criteria filter targeting the single-hop shortcut contamination documented by [53], audited for a question-length confound, and extended with a certain-control sanity set. Crucially, the identical fixed configuration was evaluated on three base models spanning the deployment spectrum: Llama 3.2 (3B, local), Llama 3.1 (8B, local), and GPT-4o-mini (cloud).

## 7.2 Answers to the Research Questions

The four questions posed in Section 1.4 can now be answered against the evidence of the preceding chapter. The answers are uneven: two hold cleanly and structurally, two hold in direction but not in magnitude, and in one case the qualification is more informative than the answer itself.

**RQ1 - Can uncertainty type be distinguished from black-box sampling geometry?** Yes, measurably but modestly, and with an important qualification. On every model, UTSR achieved the best macro F1 (0.376-0.432 on the filtered benchmark) and was the only self-answering system with non-zero aleatoric detection - the baselines sit at zero on that class by construction. The absolute separation quality, however, is far from the idealised clean fingerprint, and it is strongly conditioned on the base model: the answer to RQ1 is therefore that the fingerprints exist and are exploitable, but their sharpness is a property of the model-temperature-threshold triple, not of the questions alone.

**RQ2 - Does typed routing reduce wasted escalation relative to fixed-threshold escalation?** Yes, in direction, on all three models, and with strict dominance on the decision-relevant pairing: UTSR issued fewer wasted escalations than the fixed-threshold baseline everywhere (10 vs 14, 13 vs 21, 20 vs 22) while simultaneously missing fewer ambiguous items (5 vs 7, 2 vs 5, 1 vs 3). Moreover, every escalation UTSR issues is structured - enumerated, support-weighted interpretation options plus an audit trail - against the baseline's bare below-threshold flag. At  $n = 26$  per class the individual margins are not presented as statistically significant; the supported claim is a consistent direction across three heterogeneous models.

**RQ3 - Does typed routing eliminate unresolvable reflection on aleatoric items?** Yes, structurally. The aleatoric route contains no retrieval path, so correctly classified ambiguous items cost zero tool calls by construction; the reflection baseline burned 48-67 futile calls per model (37-42% waste) without resolving a single ambiguous item; and UTSR completed 1.8-2.4× faster than reflection on every model. This is the thesis's cleanest and most robust result. The guarantee degrades gracefully for misclassified items, bounded by the two-attempt retrieval budget.

**RQ4 - Is the mechanism robust across base models?** Partially - and the manner of the partial failure is itself the thesis's most important empirical finding. Every headline ordering is preserved at every scale (robust in trend), but per-class quality shifts dramatically and in opposite directions with model capability: aleatoric F1 falls monotonically (0.61  $\rightarrow$  0.49  $\rightarrow$  0.28) while epistemic F1 rises (0.17  $\rightarrow$  0.375  $\rightarrow$  0.475). The cluster-geometry signal is a function of the base model's sampling entropy. A weak model's noisy sampling exposes ambiguity readily but also hallucinates consistently, defeating a consistency-based certainty test; a strong model's decisive sampling collapses the polarised aleatoric fingerprint - the known weakening of sampling-based uncertainty signals as base-model consistency improves [6, 21], here observed for the type signal specifically. The mid-scale model, closest to the calibration point, performed best. The honest formulation of model-agnosticism is therefore: the mechanism transfers across models; the configuration does not, and requires per-model calibration.

## 7.3 Contributions

The work behind those answers yields five contributions, spanning the classification mechanism, the architecture built on top of it, and the evaluation framework used to test both.

1. A training-free, black-box uncertainty-type classifier for LLM agents, operating on cluster-geometry features (dominant-cluster confidence, cluster count, top-two dominance, semantic entropy) over NLI-clustered samples - to the best of my knowledge the first use of cluster count and top-two share as a runtime epistemic/aleatoric routing signal.
2. A behavioural operationalisation of reducibility, the context-injection delta test, which sidesteps definitional disputes about the epistemic/aleatoric boundary by measuring whether the admissible intervention actually reduces the uncertainty; on the model where its input geometry was healthy it resolved ten of twelve retrieval-branch entries autonomously.
3. The UTSR architecture itself - typed routes with two structural guarantees (no retrieval path on the aleatoric route; every escalation structured with enumerated options), a bounded retrieval loop with behavioural re-routing, type-tagged memory, and conservative aggregation - together with a complete open implementation on free tooling.
4. A dual failure-mode evaluation framework that scores FM1 and FM2 separately, preventing systems from trading one failure for the other invisibly, complemented by the distinction between label-level and action-level performance: on the 8B model UTSR escalated 92% of ambiguous items in structured form even though only 46% carried the correct label - for the deployment concern, the action-level number is the operative one.
5. An honestly characterised empirical result across three models, including a negative finding of independent value: the shared-clusterer false-uncertainty failure on trivially certain questions, which the baseline comparison localises to the confidence estimator rather than the routing rules by removing shortcut artifacts - evidence that the evaluation measures the construct rather than flattering the system.

## 7.4 Limitations

The claims above are bounded in six respects. They are listed in roughly the order in which they constrain interpretation of the results, from the sample size that determines what can be claimed at all, through the design choices that shaped the measured numbers, to the robustness axes that the final experimental campaign displaced and left unmeasured.

1. **Scale:** 26 items per powered class supports large-effect, direction-level conclusions across models, not fine-grained significance within one model; the certain-control set ( $n = 15$ ) is a sanity check reported with binomial intervals, not a powered comparison.
2. **Calibration transfer:** all thresholds were calibrated on the 3B model and held fixed; the observed cross-model degradation is therefore partly a calibration artifact by design, and per-model performance ceilings remain unmeasured.
3. **Clusterer brittleness:** the strict three-condition merge gate, tuned to preserve the aleatoric fingerprint, over-fragments paraphrase variation - deflating confidence on trivially certain questions (14 of 15 certain controls escalated on the 8B model, a failure shared by the threshold baseline that uses the same estimator) and plausibly driving the strong model's profile.

4. **Ground-truth noise:** HotpotQA remains an imperfect epistemic proxy even after filtering, and a residual six-word mean length difference between the classes persists after length matching; both are disclosed construct-validity threats, both biased against rather than for the system.
5. **Simulation:** retrieval is gold-context injection with guaranteed recall (real retrievers make the delta test conservative), and escalations are issued but not consumed - the usability claim for structured escalations awaits human evaluation.
6. **Untested robustness axes:** the threshold-sensitivity sweeps defined in the configuration (sample count, certainty bar, polarisation, similarity) were displaced by the cross-model campaign and remain to be run.

## 7.5 Future Work

The findings define a concrete agenda, ordered here by leverage. Each item is a repair or extension that the evaluation itself identified - a component whose failure was observed, a design choice whose alternative is now specified, or a measurement the campaign deferred - rather than a generic call for further research.

**Paraphrase-tolerant clustering:** The single highest-leverage repair identified by the evaluation is the merge gate. Candidate directions include answer normalisation before clustering (extracting the minimal answer span from verbose responses), asymmetric bidirectional entailment in place of the similarity floor, and per-model calibration of the gate against a held-out paraphrase set. Success is directly measurable against the certain-control set, whose current 0/15-3/15 pass rates leave unambiguous headroom.

**Per-model calibration against measured sampling entropy:** The cross-model finding suggests a cheap procedure: probe the base model’s raw sampling entropy on a small fixed question set, then set temperature, sample count, and the geometry thresholds from that measurement - effectively normalising the classifier’s input distribution before the fixed rules apply. The configuration’s sweep grids provide the search space; the three-model results provide the validation targets, with the prediction that calibration recovers aleatoric detection on strong models and suppresses false certainty on weak ones.

**A relative or post-hoc delta criterion:** Replacing the additive gain test ( $\Delta \geq 0.10$ ) with a relative-gain or post-injection-certainty criterion would decouple the reducibility decision from the pre-injection baseline, removing the ceiling effect that starved the retrieval branch under the fragmented distributions of the strongest model.

**Scale-up and inference:** The paired analyses deferred in Section 6.6 - McNemar’s exact test on the escalation counts and bootstrap intervals on the F1 differences - can be run directly on the serialised per-item traces, and are the immediate next step. Growing the powered classes toward 100+ items per class - a compute budget question, at roughly 1.5-5.5 hours per model per full run at the current size - would give those tests the power to resolve the FM1 margins that the present direction-level claims leave open, and would permit the deferred threshold-sensitivity sweeps (RQ4 in its original single-model form).

**Benchmark breadth:** Adding SituatedQA [48] as a second aleatoric source would test generalisation beyond AmbigNQ’s ambiguity mix to explicitly temporal and geographic under-specification, and MuSiQue-style shortcut-free multi-hop data would strengthen the epistemic class beyond what filtering HotpotQA can achieve.

**Real retrieval and real reviewers:** Replacing gold-context injection with an actual retriever [33] tests the delta test under imperfect recall, where its conservatism becomes a feature; a small user study measuring reviewer resolution time on structured versus unstructured escalations would convert the FM1-quality argument from design claim to measured effect.

**Beyond question answering:** The routing mechanism inspects only the shape of an agent’s output distribution, not the task; transplanting it to workflow subtasks that are not natural-language questions - compliance checks, extraction steps, triage decisions - is the natural test of the pipeline-orchestration framing, with the type-tagged memory and conservative aggregation already in place for multi-subtask composition.

## 7.6 Closing Remarks

The central claim of this thesis is deliberately modest in form and, I believe, correct in substance: what kind of uncertainty an agent has should determine what the pipeline does next, and that kind is partially legible, at runtime and without any training, in the shape of the agent’s own sampled disagreement. The evaluation shows both halves honestly. The typed routes deliver exactly the structural properties they were designed for - zero retrieval on detected ambiguity, structured escalations everywhere, immunity to the unresolvable reflection that consumes 40% of a type-blind loop’s budget - and they do so on every model tested. At the same time, the signal those routes depend on is not free: it is borrowed from the base model’s sampling entropy, it weakens as models grow more decisive, and it passes through a clustering step whose calibration is the true load-bearing component of the whole system. A thesis that had reported only the wins would have missed the finding that matters most for whoever builds the next iteration: in sampling-based uncertainty typing, the classifier’s rules are the easy part - making the distribution they read comparable across models is the research problem. This work provides the architecture, the evaluation framework, the failure analysis, and the calibration agenda for solving it.

# Appendix A - Core Implementation

This appendix reproduces the components of the implementation that embody the thesis' central contribution - uncertainty-type-driven specialist-agent routing: the calibrated configuration (A.1), the semantic clusterer whose merge gate is analysed in Section 6.4 (A.2), the uncertainty-type decision rules (A.3), the typed routing contracts (A.4), the routing loop with the context-injection delta test (A.5), and the retrieval simulation and clarification agent (A.6). Orchestration scripts, baseline harnesses, plotting utilities, the reflection-cycle detector, the remaining specialist agents, and the result files are omitted here; the complete implementation, evaluation harness, and result files are available in the accompanying repository at <https://github.com/Hymavathy/Agent>.

## A.1 Hyperparameters and Thresholds

```
1  """
2  """
3  config.py - Tunable parameters for uncertainty-routing experiments.
4  """
5
6  # --- Sampling ---
7
8  # Reduced samples for practical runtime on local models:
9  # 8 samples gives good signal while keeping runtime manageable.
10 # NOTE on entropy scale: semantic entropy H is computed in BITS (
11     log base 2,
12 # see clusterer.compute_entropy / uncertainty_types.compute_entropy
13     ). With
14 # N_SAMPLES = 8 the maximum possible H is log2(8) = 3.0 bits, which
15     is why the
16 # LOW/HIGH entropy thresholds below (2.00 / 2.50) are on a 0-3 bit
17     scale.
18 N_SAMPLES = 8
19 TEMPERATURE = 0.60
20
21 # --- Classification thresholds ---
22
23 # confidence of largest cluster - calibrated for local models (
24     llama3.2)
25 # 0.80 was too high for 3B models - they rarely achieve that
26     agreement
27 CONFIDENCE_THRESHOLD = 0.65
28
29 # entropy thresholds - RELAXED for aleatoric detection
30 # Aleatoric questions often have moderate entropy (2-3
31     interpretations)
32 LOW_ENTROPY_THRESHOLD = 2.00
33 HIGH_ENTROPY_THRESHOLD = 2.50
34
```

```

28 # top-2 cluster dominance - LOWERED to catch more aleatoric
    patterns
29 # Ambiguous questions often split 60/40 or 50/30/20, not just 80/20
30 POLARISE_THRESHOLD = 0.60
31
32 # epistemic cluster count - RAISED to avoid false epistemic
    classification
33 MIN_CLUSTERS_EPISTEMIC = 5
34
35 # retrieval improvement threshold
36 DELTA_THRESHOLD = 0.10
37
38 # --- Semantic clustering ---
39
40 NLI_MODEL = "cross-encoder/nli-deberta-v3-small"
41
42 # IMPORTANT:
43 # 0.25 was massively over-merging unrelated answers
44 # 0.60 was still over-merging distinct interpretations of ambiguous
    questions
45 # 0.70 gives better separation - distinct answers stay in separate
    clusters
46 SIMILARITY_THRESHOLD = 0.70
47
48 # contradiction threshold - LOWERED to block more merges
49 # Even mild contradiction should prevent merging
50 CONTRADICTION_THRESHOLD = 0.35
51
52 # entailment threshold - RAISED to require stronger agreement for
    merging
53 ENTAILMENT_THRESHOLD = 0.55
54
55 # --- Dataset ---
56
57 N_EPISTEMIC_SAMPLES = 26
58 N_ALEATORIC_SAMPLES = 26
59 RANDOM_SEED = 42
60
61 # --- Experiment ---
62
63 MAX_TOOL_CALLS_BASELINE = 3
64 MAX_RETRIEVAL_ATTEMPTS = 2
65
66 # Output folders, one per evaluated model backend.
67 # The 26q experiment runners (run_26q.py / run_26q_filtered.py)
    define their own
68 # per-model output dirs (results_26q_*), but these three cover the
    base runs.

```

```

69 OUTPUT_DIR = "results_3b/"          # Llama 3.2 (3B, local via
    Ollama)
70 OUTPUT_DIR_8B = "results_8b/"      # Llama 3.1 (8B, local via
    Ollama)
71 OUTPUT_DIR_GPT4MINI = "results_gpt4mini/" # GPT-4o-mini (cloud
    via OpenAI)
72
73 # --- Sensitivity sweeps ---
74
75 N_SAMPLES_SWEEP = [5, 10, 15, 20]
76
77 CONFIDENCE_THRESH_SWEEP = [0.50, 0.60, 0.70, 0.80]
78
79 POLARISE_THRESH_SWEEP = [0.65, 0.75, 0.80, 0.90]
80
81 SIMILARITY_THRESH_SWEEP = [0.50, 0.55, 0.60, 0.65, 0.70]

```

Listing A.1: config.py (full). All tunable thresholds referenced throughout Appendix A and Chapter 4; reproducing the reported results requires exactly these values.

Thresholds were calibrated empirically against a 3B-parameter local model (Llama 3.2) rather than derived analytically; the in-line comments record the values that were tried and rejected (e.g. CONFIDENCE\_THRESHOLD was lowered from 0.80 to 0.65, SIMILARITY\_THRESHOLD raised from 0.25 to 0.70). Every threshold used by `classify_uncertainty_type` (Listing A.3) and `cluster_outputs` (Listing A.2) is defined here, so this listing is placed first and referenced by name throughout the remaining listings.

## A.2 Semantic Clustering

```

1
2 """
3 classifier/clusterer.py
4
5 Improved semantic clustering for uncertainty routing.
6 """
7
8 import math
9 import torch
10
11 from sentence_transformers import (
12     SentenceTransformer,
13     CrossEncoder,
14     util
15 )
16
17 from config import (
18     SIMILARITY_THRESHOLD,

```

```

19     NLI_MODEL,
20     CONTRADICTION_THRESHOLD,
21     ENTAILMENT_THRESHOLD
22 )
23
24 _model = None
25 _nli_model = None
26
27 def _get_model():
28     global _model
29
30     if _model is None:
31         print("Loading embedding model...")
32         _model = SentenceTransformer("all-mpnet-base-v2")
33
34     return _model
35
36 def _get_nli_model():
37     global _nli_model
38
39     if _nli_model is None:
40         print("Loading NLI model...")
41         _nli_model = CrossEncoder(NLI_MODEL)
42
43     return _nli_model
44
45 def _nli_scores(
46     nli: CrossEncoder,
47     a: str,
48     b: str
49 ):
50     """
51     Returns:
52         contradiction, neutral, entailment
53     """
54
55     scores = nli.predict([(a, b)])
56
57     contradiction = float(scores[0][0])
58     neutral        = float(scores[0][1])
59     entailment     = float(scores[0][2])
60
61     return contradiction, neutral, entailment
62
63 def _should_merge(
64     sim: float,
65     contradiction: float,
66     entailment: float

```

```

67 ) -> bool:
68     """
69     Conservative merge criterion.
70     """
71
72     if sim < SIMILARITY_THRESHOLD:
73         return False
74
75     if contradiction > CONTRADICTION_THRESHOLD:
76         return False
77
78     if entailment < ENTAILMENT_THRESHOLD:
79         return False
80
81     return True
82
83 def cluster_outputs(outputs: list[str]) -> list[list[str]]:
84     """
85     Semantic clustering with:
86     - cosine similarity
87     - NLI contradiction filtering
88     - entailment gating
89     """
90
91     if not outputs:
92         return []
93
94     model = _get_model()
95     nli = _get_nli_model()
96
97     embeddings = model.encode(
98         outputs,
99         convert_to_tensor=True
100     )
101
102     clusters = []
103
104     for emb, text in zip(embeddings, outputs):
105
106         best_idx = None
107         best_sim = -1.0
108
109         for idx, (texts, centroid, embs) in enumerate(clusters):
110
111             sim = util.cos_sim(
112                 emb,
113                 centroid
114             ).item()

```

```

115
116         contradiction, neutral, entailment = _nli_scores(
117             nli,
118             texts[0],
119             text
120         )
121
122         if _should_merge(
123             sim,
124             contradiction,
125             entailment
126         ):
127             if sim > best_sim:
128                 best_sim = sim
129                 best_idx = idx
130
131         if best_idx is not None:
132
133             texts, centroid, embs = clusters[best_idx]
134
135             texts.append(text)
136             embs.append(emb)
137
138             new_centroid = torch.stack(embs).mean(dim=0)
139
140             clusters[best_idx][1] = new_centroid
141
142         else:
143
144             clusters.append([
145                 [text],
146                 emb.clone(),
147                 [emb]
148             ])
149
150         final_clusters = [
151             c[0]
152             for c in clusters
153         ]
154
155         final_clusters.sort(
156             key=len,
157             reverse=True
158         )
159
160         return final_clusters
161
162     def compute_semantic_confidence(

```

```

163     clusters: list[list[str]],
164     n: int
165 ) -> float:
166
167     if not clusters:
168         return 0.0
169
170     return len(clusters[0]) / n
171
172 def compute_entropy(
173     clusters: list[list[str]],
174     n: int
175 ) -> float:
176     """
177     Semantic entropy in BITS. Delegates to the single canonical
178     implementation
179     in agent.uncertainty_types.compute_entropy (kept here for
180     backward-compatibility; classifier code imports it from this
181     module).
182     """
183     from agent.uncertainty_types import compute_entropy as
184         _canonical_entropy
185     return _canonical_entropy(clusters, n)

```

Listing A.2: clusterer.py (classifier/clusterer.py). Turns N raw LLM samples into semantic clusters; implements the clustering step described in Section 4.2 and supplies `compute_semantic_confidence` (T1) and `compute_entropy` (H) used by the classifier in Listing A.3.

`_should_merge` is the conservative merge gate: two samples are only merged into the same cluster if their cosine similarity clears `SIMILARITY_THRESHOLD`, their NLI contradiction score stays below `CONTRADICTION_THRESHOLD`, and their entailment score clears `ENTAILMENT_THRESHOLD` — similarity alone was found to over-merge semantically distinct answers. `cluster_outputs` then grows clusters online, comparing each new sample against every existing cluster centroid and merging into the best-scoring candidate rather than re-clustering from scratch.

### A.3 Uncertainty Classifier

```

1
2 """
3 classifier/pattern_detector.py
4
5 Improved uncertainty classifier using:
6 - semantic entropy
7 - cluster geometry

```

```

8 - top-two dominance
9 """
10
11 from dataclasses import dataclass
12
13 from classifier.clusterer import (
14     cluster_outputs,
15     compute_semantic_confidence,
16     compute_entropy
17 )
18
19 from config import (
20     CONFIDENCE_THRESHOLD,
21     LOW_ENTROPY_THRESHOLD,
22     HIGH_ENTROPY_THRESHOLD,
23     POLARISE_THRESHOLD,
24     MIN_CLUSTERS_EPISTEMIC
25 )
26
27 class ClassificationResult:
28
29     uncertainty_type: str
30
31     confidence_score: float
32
33     entropy: float
34
35     n_clusters: int
36
37     top_two_share: float
38
39     clusters: list
40
41     reasoning: str
42
43
44 def classify_uncertainty_type(
45     outputs: list[str]
46 ) -> ClassificationResult:
47     """
48     Classify uncertainty type from sampled LLM outputs using
49         cluster geometry.
50
51     Decision tree (order matters):
52         1. CERTAIN: confidence >= threshold (dominant single
53             cluster)
54         2. ALEATORIC: top-2 clusters dominate AND entropy is bounded

```

```

53         (few stable interpretations - question is
54             ambiguous)
55     3. EPISTEMIC: high entropy OR many scattered clusters
56                 (model lacks knowledge - answers are random)
57     4. MIXED:      fallback - treated as aleatoric (conservative)
58
59     Key insight: aleatoric uncertainty produces STRUCTURED
60                 disagreement
61                 (2-3 coherent answer groups), while epistemic produces
62                 SCATTERED
63                 disagreement (many singleton/small clusters, high entropy).
64     """
65
66     n = len(outputs)
67
68     clusters = cluster_outputs(outputs)
69
70     confidence = compute_semantic_confidence(
71         clusters,
72         n
73     )
74
75     entropy = compute_entropy(
76         clusters,
77         n
78     )
79
80     n_clusters = len(clusters)
81
82     # top-two share (computed early - used in multiple branches)
83     top_two_count = sum(
84         len(c)
85         for c in clusters[:2]
86     )
87     top_two_share = top_two_count / n if n > 0 else 0.0
88
89     # ---
90     # CERTAIN
91     # Single dominant cluster with high confidence
92     # ---
93
94     if confidence >= CONFIDENCE_THRESHOLD:
95
96         return ClassificationResult(
97             uncertainty_type="certain",
98             confidence_score=confidence,
99             entropy=entropy,
100             n_clusters=n_clusters,

```

```

98         top_two_share=top_two_share,
99         clusters=clusters,
100         reasoning=(
101             f"High confidence "
102             f"({confidence:.2f} >= {CONFIDENCE_THRESHOLD})"
103         )
104     )
105
106     # ---
107     # ALEATORIC
108     # Structured disagreement: few clusters dominate
109     # Key signals:
110     #   - Top-2 clusters capture most samples (polarised)
111     #   - Entropy is bounded (not scattered chaos)
112     #   - Cluster count is small (2-4 interpretations)
113     # ---
114
115     # Primary aleatoric signal: polarised top-2 with bounded
116     # entropy
117     is_polarised = top_two_share >= POLARISE_THRESHOLD
118     is_bounded_entropy = entropy <= LOW_ENTROPY_THRESHOLD
119     is_few_clusters = n_clusters <= 4
120
121     # Secondary aleatoric signal: exactly 2-3 clusters with
122     # the top cluster not dominant enough to be "certain"
123     is_binary_split = (
124         n_clusters in (2, 3)
125         and confidence < CONFIDENCE_THRESHOLD
126         and top_two_share >= 0.75
127     )
128
129     # Tertiary aleatoric signal: 2-5 clusters with moderate top-2
130     # share
131     # Catches AmbigNQ-style questions where the model produces a
132     # small number
133     # of distinct interpretations but with moderate (not high)
134     # polarisation.
135     # Without this, these questions fall through to epistemic due
136     # to entropy.
137     is_structured_few = (
138         n_clusters in (2, 3, 4, 5)
139         and top_two_share >= 0.50
140         and confidence < CONFIDENCE_THRESHOLD
141     )
142
143     # Quaternary aleatoric signal: few clusters (k <= 4) that are
144     # polarised

```

```

139 # (top-2 share >= POLARISE_THRESHOLD, i.e. 0.60) but whose
      entropy is NOT
140 # bounded (H > LOW_ENTROPY_THRESHOLD). This is the (
      is_few_clusters and
141 # is_polarised) term below. It fires for k <= 4 with top-2 in
      [0.60, 0.75)
142 # and entropy > 2.0 bits - a region none of the three signals
      above catch
143 # (signal 1 requires bounded entropy, signal 2 requires top-2
      >= 0.75,
144 # signal 3 fires from top-2 >= 0.50 but is otherwise subsumed
      here).
145 if (is_polarised and is_bounded_entropy) or is_binary_split or
      (is_few_clusters and is_polarised) or is_structured_few:
146
147     return ClassificationResult(
148         uncertainty_type="aleatoric",
149         confidence_score=confidence,
150         entropy=entropy,
151         n_clusters=n_clusters,
152         top_two_share=top_two_share,
153         clusters=clusters,
154         reasoning=(
155             f"Structured disagreement detected. "
156             f"top2={top_two_share:.2f}, H={entropy:.2f}, "
157             f"K={n_clusters}. Multiple stable interpretations."
158         )
159     )
160
161 # ---
162 # EPISTEMIC
163 # Scattered disagreement: many clusters, high entropy
164 # Model is guessing - knowledge gap
165 # ---
166
167 if (
168     entropy >= HIGH_ENTROPY_THRESHOLD
169     or n_clusters >= MIN_CLUSTERS_EPISTEMIC
170 ):
171
172     return ClassificationResult(
173         uncertainty_type="epistemic",
174         confidence_score=confidence,
175         entropy=entropy,
176         n_clusters=n_clusters,
177         top_two_share=top_two_share,
178         clusters=clusters,
179         reasoning=(

```

```

180         f"Scattered uncertainty. "
181         f"H={entropy:.2f}, K={n_clusters}. "
182         f"Model lacks stable knowledge."
183     )
184 )
185
186 # ---
187 # MIXED      treated as ALEATORIC (conservative)
188 # Better to clarify than to waste retrieval calls
189 # ---
190
191 return ClassificationResult(
192     uncertainty_type="aleatoric",
193     confidence_score=confidence,
194     entropy=entropy,
195     n_clusters=n_clusters,
196     top_two_share=top_two_share,
197     clusters=clusters,
198     reasoning=(
199         f"Borderline pattern (H={entropy:.2f}, K={n_clusters}),
200         "
201         f"top2={top_two_share:.2f}). "
202         f"Conservatively routing as aleatoric."
203     )
204 )
205
206 def build_structured_escalation(
207     question: str,
208     clusters: list[list[str]],
209     conf: float
210 ):
211
212     total = sum(
213         len(c)
214         for c in clusters
215     )
216
217     options = []
218
219     for i, cluster in enumerate(clusters[:3]):
220
221         options.append({
222             "option": chr(65 + i),
223             "answer": cluster[0],
224             "support_count": len(cluster),
225             "support_fraction":
226                 f"{len(cluster)/total:.0%}"
227         })

```

```

227
228     return {
229         "status": "requires_human_decision",
230         "question": question,
231         "uncertainty_type": "aleatoric",
232         "confidence": conf,
233         "message": (
234             "Question has multiple valid "
235             "interpretations."
236         ),
237         "options": options,
238         "action_required": (
239             "Select the correct interpretation."
240         )
241     }

```

Listing A.3: pattern\_detector.py (classifier/pattern\_detector.py). Implements the CERTAIN / ALEATORIC / EPISTEMIC decision rule described in Section 4.2.2; thresholds are defined in Listing A.1, cluster geometry is supplied by Listing A.2.

The ordering of branches matters: the CERTAIN check runs first so a dominant cluster short-circuits before the polarisation tests are evaluated. The `is_structured_few` branch is what captures moderate AmbigNQ-style splits (2–5 clusters, top-two share as low as 0.50) that would otherwise fall through to EPISTEMIC purely on entropy; without it, genuinely ambiguous-but-not-highly-polarised questions were being misrouted to retrieval. The final “MIXED” fallback is deliberately conservative — borderline cases are routed to clarification rather than retrieval, since an unnecessary clarification costs one round-trip whereas an unnecessary retrieval call risks reinforcing a wrong interpretation.

## A.4 Uncertainty Signal and Routing Types

```

1  import math
2  from dataclasses import dataclass, field
3  from enum import Enum
4  from typing import Optional
5
6  # --- Enums ---
7
8  class UncertaintyType(Enum):
9      CERTAIN = "certain" # High confidence - majority cluster
10         dominates
11      EPISTEMIC = "epistemic" # Reducible - model lacks knowledge;
12         RAG can help
13      ALEATORIC = "aleatoric" # Irreducible - genuinely ambiguous;
14         only clarif. helps

```

```

13 class RoutingDecision(Enum):
14     ANSWER = "answer" # Return directly -> AnswerAgent
15     RETRIEVE = "retrieve" # Fetch context -> RetrievalAgent
16     CLARIFY = "clarify" # Structure options ->
17         ClarificationAgent
18     ESCALATE = "escalate" # Last-resort handoff->
19         HumanEscalationAgent
20
21 # --- Signal dataclass ---
22
23 class UncertaintySignal:
24     uncertainty_type: UncertaintyType
25     routing_decision: RoutingDecision
26     semantic_entropy: float # H over cluster distribution
27     cluster_count: int # K = number of semantic
28         clusters
29     top_two_share: float # T2 = (|C1| + |C2|) / N
30     confidence_score: float # T1 = |C1| / N (largest cluster
31         fraction)
32     raw_samples: list[str]
33     structured_options: list[str] = field(default_factory=list)
34
35 def compute_entropy(clusters: list[list[str]], n: int) -> float:
36     if n == 0:
37         return 0.0
38     return -sum(
39         (p := len(c) / n) * math.log2(p)
40         for c in clusters
41         if (p := len(c) / n) > 0
42     )

```

Listing A.4: uncertainty\_types.py (agent/uncertainty\_types.py). Typed contracts shared by the classifier and router, and the closed-form semantic entropy  $H = -\sum_i p_i \log_2(p_i)$  referenced as Eq. 4.1 in the main text.

UncertaintyType and RoutingDecision replace raw strings used elsewhere in the codebase, so a typo in a routing label is caught at import time rather than silently producing an incorrect metric. UncertaintySignal is the single object threaded between the classifier and the specialist agents (Listing A.6); it carries the raw T1 (confidence\_score), T2 (top\_two\_share) and H (semantic\_entropy) statistics referenced in Section 4.3 so no recomputation is needed downstream.

## A.5 Router

```

1
2 from typing import Optional
3

```

```

4 from agent.llm_agent import (
5     ToolEquippedAgent,
6     AnswerAgent,
7     RetrievalAgent,
8     ClarificationAgent,
9     HumanEscalationAgent,
10 )
11 from agent.uncertainty_types import (
12     UncertaintyType,
13     RoutingDecision,
14     UncertaintySignal,
15     RoutingResult,
16     compute_entropy,
17 )
18 from classifier.sampler import sample_with_early_stop as
19     sample_agent
20 from classifier.clusterer import cluster_outputs,
21     compute_semantic_confidence
22 from classifier.pattern_detector import (
23     classify_uncertainty_type,
24     build_structured_escalation,
25 )
26 from pipeline.memory import PipelineMemory, MemoryEntry
27 from config import (
28     CONFIDENCE_THRESHOLD,
29     N_SAMPLES,
30     TEMPERATURE,
31     DELTA_THRESHOLD,
32     MAX_RETRIEVAL_ATTEMPTS,
33 )
34
35 # --- Uncertainty type -> enum mapping ---
36
37 _TYPE_MAP: dict[str, UncertaintyType] = {
38     "confident": UncertaintyType.CERTAIN,
39     "certain": UncertaintyType.CERTAIN,
40     "epistemic": UncertaintyType.EPISTEMIC,
41     "aleatoric": UncertaintyType.ALEATORIC,
42     "mixed": UncertaintyType.ALEATORIC, # conservative: treat
43         as aleatoric
44 }
45
46 def _to_routing_decision(unc_type: UncertaintyType) ->
47     RoutingDecision:
48     return {
49         UncertaintyType.CERTAIN: RoutingDecision.ANSWER,
50         UncertaintyType.EPISTEMIC: RoutingDecision.RETRIEVE,
51         UncertaintyType.ALEATORIC: RoutingDecision.CLARIFY,

```

```

48     }[unc_type]
49
50 # --- UncertaintyRouter ---
51
52 class UncertaintyRouter:
53
54     # --- Internal: classify one round ---
55
56     def _classify(
57         self, question: str, context: str = ""
58     ) -> tuple[list[list[str]], UncertaintySignal]:
59         """
60         Sample + cluster + classify. Returns (clusters, signal).
61         """
62         samples = sample_agent(
63             self.base_agent, question,
64             context=context,
65             n=self.n_samples,
66             temperature=self.temperature,
67         )
68         clusters = cluster_outputs(samples)
69         cr = classify_uncertainty_type(samples) # uses
70             pattern_detector
71         unc_type = _TYPE_MAP.get(cr.uncertainty_type,
72             UncertaintyType.ALEATORIC)
73         routing = _to_routing_decision(unc_type)
74         entropy = compute_entropy(clusters, len(samples))
75         conf = compute_semantic_confidence(clusters, len(
76             samples))
77
78         n = len(samples)
79         sizes = sorted([len(c) for c in clusters], reverse=True
80             )
81         t2 = (sizes[0] + (sizes[1] if len(sizes) > 1 else 0)
82             ) / n if n else 0.0
83
84         signal = UncertaintySignal(
85             uncertainty_type = unc_type,
86             routing_decision = routing,
87             semantic_entropy = entropy,
88             cluster_count = len(clusters),
89             top_two_share = t2,
90             confidence_score = conf,
91             raw_samples = samples,
92             structured_options = [c[0] for c in clusters],
93         )
94         return clusters, signal
95

```

```

91 # --- Public: route one question ---
92
93 def route(self, question: str) -> RoutingResult:
94
95     self.base_agent.reset_tool_counter()
96     tool_calls = 0
97     current_ctx = "" # grows if retrieval finds
98     something
99     audit_log: list[str] = []
100
101     # --- Round 0: initial classification (no context) ---
102     clusters, signal = self._classify(question, context="")
103     tool_calls += 1 # count the sampling round as one "call"
104
105     # --- CERTAIN AnswerAgent ---
106     if signal.routing_decision == RoutingDecision.ANSWER:
107         answer = self.answer_agent.run(clusters)
108         return RoutingResult(
109             question = question,
110             signal = signal,
111             predicted_type = signal.uncertainty_type.value,
112             routing_decision = signal.routing_decision.value,
113             agent_used = "AnswerAgent",
114             tool_calls_made = tool_calls,
115             final_answer = answer,
116             escalated = False,
117         )
118
119     # --- ALEATORIC ClarificationAgent (FM2: skip retrieval
120     # entirely)
121     if signal.routing_decision == RoutingDecision.CLARIFY:
122         structured = self.clarify_agent.format(
123             question, clusters, signal.confidence_score
124         )
125         return RoutingResult(
126             question = question,
127             signal = signal,
128             predicted_type = signal.uncertainty_type.value,
129             routing_decision = signal.routing_decision.value,
130             agent_used = "ClarificationAgent",
131             tool_calls_made = tool_calls,
132             escalation_message = structured,
133             escalated = True,
134         )
135
136     # --- EPISTEMIC RetrievalAgent loop ---
137     # Each iteration: retrieve context delta test re-
138     classify.

```

```

136         # If delta > DELTA_THRESHOLD: epistemic RESOLVED
137         AnswerAgent.
138
139         # If loop exhausted without resolution:
140         HumanEscalationAgent.
141
142
143         last_clusters = clusters
144         last_signal = signal
145         conf_pre = signal.confidence_score
146
147         for attempt in range(self.max_retrieval_attempts):
148             audit_log.append(f"Attempt {attempt + 1}: retrieving
149                             context...")
150
151             retrieved = self.retrieval_agent.retrieve(
152                 self.base_agent, question, self.context
153             )
154
155             if retrieved is None:
156                 audit_log.append("No context retrieved.
157                                 Escalating.")
158                 break # nothing available; go straight to human
159
160             tool_calls += 1 # count each retrieval as a tool call
161             current_ctx = retrieved
162             audit_log.append(f"Context retrieved ({len(
163                             retrieved)} chars).")
164
165             # Re-classify WITH context - delta test
166             clusters_post, signal_post = self._classify(question,
167                                                         context=current_ctx)
168             tool_calls += 1
169             conf_post = signal_post.confidence_score
170             delta = conf_post - conf_pre
171
172             audit_log.append(
173                 f"conf_pre={conf_pre:.2f} conf_post={
174                     conf_post:.2f}"
175                 f"delta={delta:.2f} (threshold={self.
176                     delta_threshold})"
177             )
178
179             if delta >= self.delta_threshold:
180                 # --- Epistemic RESOLVED ---
181                 answer = self.answer_agent.run(clusters_post)
182                 audit_log.append(f"RESOLVED by retrieval.
183                                 Answer: {answer}")
184                 return RoutingResult(
185                     question, answer, question,

```

```

175         signal                = signal_post,
176         predicted_type         = "epistemic",
177         routing_decision       = RoutingDecision.ANSWER.
178             value,
179         agent_used              = "
180             RetrievalAgentAnswerAgent ",
181         tool_calls_made        = tool_calls,
182         final_answer            = answer,
183         retrieved_context       = current_ctx,
184         escalated                = False,
185     )
186
187     # Context didn't help enough - check if pattern changed
188     last_clusters = clusters_post
189     last_signal    = signal_post
190     conf_pre       = conf_post
191
192     if signal_post.routing_decision == RoutingDecision.
193         CLARIFY:
194         # Context injection turned it aleatoric - clarify
195         instead
196         audit_log.append("          Pattern now ALEATORIC.
197             Routing to ClarificationAgent.")
198         structured = self.clarify_agent.format(
199             question, last_clusters, last_signal.
200             confidence_score
201         )
202         return RoutingResult(
203             question                = question,
204             signal                  = last_signal,
205             predicted_type           = "aleatoric",
206             routing_decision         = RoutingDecision.CLARIFY.
207                 value,
208             agent_used               = "
209                 RetrievalAgentClarificationAgent ",
210             tool_calls_made          = tool_calls,
211             escalation_message       = structured,
212             retrieved_context         = current_ctx,
213             escalated                 = True,
214         )
215
216     # --- Epistemic UNRESOLVED -> HumanEscalationAgent ---
217     escalation = self.human_agent.format(
218         question, last_clusters, last_signal.confidence_score,
219         audit_log
220     )
221     return RoutingResult(
222         question                = question,

```

```

214         signal                = last_signal ,
215         predicted_type         = "epistemic",
216         routing_decision       = RoutingDecision.ESCALATE.value ,
217         agent_used              = "HumanEscalationAgent",
218         tool_calls_made         = tool_calls ,
219         escalation_message      = escalation ,
220         retrieved_context       = current_ctx or None ,
221         escalated               = True ,
222     )
223
224     # --- Write to pipeline memory ---
225     memory.write(MemoryEntry(
226         node_id                = node_id ,
227         question                 = question ,
228         output                   = (result.final_answer or result.
229             escalation_message) ,
229         confidence              = result.signal.confidence_score ,
230         uncertainty_type         = result.predicted_type ,
231         tool_calls_used          = result.tool_calls_made ,
232         resolved                 = not result.escalated ,
233     ))
234
235     return result.to_result_dict(ground_truth_type)

```

Listing A.5: router.py (pipeline/router.py), UncertaintyRouter class. Implements the routing loop, the retrieval delta test, and epistemic escalation; the run\_your\_system() evaluation-harness wrapper is omitted here as it contains no methodological content.

route() (near the end of the listing) is the entry point and directly encodes the two failure modes the thesis addresses: the ALEATORIC branch returns from ClarificationAgent immediately, so an aleatoric question can never reach RetrievalAgent (FM1 elimination — the retrieval loop below it is simply unreachable for that branch). The EPISTEMIC branch implements the delta test: after each retrieval attempt the question is re-classified with the new context, and only a confidence gain of at least DELTA\_THRESHOLD (Listing A.1) is accepted as “resolved”; a post-retrieval reclassification to ALEATORIC is itself treated as a routing outcome rather than a failure, since the added context revealed genuine ambiguity rather than a knowledge gap.

## A.6 Specialist Agents

```

1
2 import os
3 import time
4 from typing import Optional
5 from openai import OpenAI
6 from dotenv import load_dotenv
7

```

```

8 load_dotenv()
9
10 PROVIDER_CONFIGS = {
11     "ollama": {
12         "base_url": "http://localhost:11434/v1",
13         "api_key": "ollama",
14         "model": "llama3.2",
15         "description": "Local Llama 3.2 (3B) via Ollama - free, no
            rate limits"
16     },
17     "openai": {
18         "base_url": "https://api.openai.com/v1",
19         "api_key": os.getenv("OPENAI_API_KEY", ""),
20         "model": "gpt-4o-mini",
21         "description": "OpenAI GPT-4o-mini - high quality, pay-per-
            token"
22     },
23 }
24
25 PROVIDER = os.getenv("LLM_PROVIDER", "ollama").lower()
26
27 # --- Core LLM wrapper (sampler) ---
28
29 class ToolEquippedAgent:
30     """
31     Tool-equipped LLM agent. Works with Ollama or OpenAI.
32     Change LLM_PROVIDER in .env to switch - no code changes needed.
33
34     Role in the routing pipeline: SAMPLER only.
35     The specialist agents (below) handle the routing decisions.
36     """
37
38     def __init__(self):
39         config = PROVIDER_CONFIGS[PROVIDER]
40         self.client = OpenAI(
41             api_key=config["api_key"],
42             base_url=config["base_url"]
43         )
44         self.model = config["model"]
45         self.tool_calls_made = 0
46         print(f"Using: {config['description']}")
47         print(f"Model: {self.model}")
48
49     def answer(
50         self,
51         question: str,
52         context: str = "",
53         temperature: float = 0.7

```

```

54     ) -> str:
55         """
56         Ask the agent a question, optionally with context injected.
57         Returns a single string answer.
58
59         Args:
60             question:    the question to answer
61             context:      supporting context (simulates tool result)
62             temperature: sampling temperature
63         """
64         system_prompt = (
65             "You are a helpful assistant. Answer the question
66             concisely "
67             "in one sentence. If you are uncertain, give your best
68             answer anyway."
69         )
70
71         user_message = question
72         if context:
73             user_message = (
74                 f"Use the following information to answer:\n\n"
75                 f"{context}\n\n"
76                 f"Question: {question}"
77             )
78
79         response = self.client.chat.completions.create(
80             model=self.model,
81             messages=[
82                 {"role": "system", "content": system_prompt},
83                 {"role": "user", "content": user_message}
84             ],
85             temperature=temperature,
86             max_tokens=150 # answer-length cap (see Table 4.4 /
87                             config note)
88         )
89
90         return response.choices[0].message.content.strip()
91
92     def simulate_tool_call(self, question: str, context: str) ->
93         str:
94             """
95             Simulate a tool/retrieval call.
96             In experiments: returns the HotpotQA supporting context.
97             In production: this would call Wikipedia search / RAG /
98             database.
99
100             Tracks how many tool calls are made (for cost reporting).
101             """

```

```

97         self.tool_calls_made += 1
98
99         if context:
100             return context # simulate retrieval returning relevant
                               docs
101
102         # If no context available (aleatoric cases), simulate empty
           retrieval
103         return "No additional relevant information found."
104
105     def reset_tool_counter(self):
106         self.tool_calls_made = 0
107
108 class ClarificationAgent:
109
110     def format(
111         self,
112         question: str,
113         clusters: list[list[str]],
114         confidence: float
115     ) -> dict:
116         """
117         Build a structured clarification request from the top
           clusters.
118
119         Args:
120             question: the original ambiguous question
121             clusters: semantically clustered LLM samples, sorted
                       by size desc
122             confidence: semantic confidence of the top cluster
123
124         Returns:
125             Structured dict with options - same schema as
               build_structured_escalation()
126             for backward compatibility with metrics.py.
127         """
128         sorted_c = sorted(clusters, key=len, reverse=True)
129         total = sum(len(c) for c in sorted_c)
130         options = []
131         for i, cluster in enumerate(sorted_c[:3]): # top 3
132             interpretations max
133             options.append({
134                 "option": chr(65 + i), # A, B, C
135                 "answer": cluster[0],
136                 "support_count": len(cluster),
137                 "support_fraction": f"{len(cluster)/total:.0%}"
138             })
139
140     return {
141         "status": "requires_clarification",

```

```

139         "question":          question,
140         "uncertainty_type":   "aleatoric",
141         "confidence":         confidence,
142         "message": (
143             "ALEATORIC UNCERTAINTY DETECTED\n"
144             f"This question has {len(sorted_c)} valid
145             interpretations.\n"
146             "Please specify which interpretation applies to
147             your context."
148         ),
149         "options":            options,
150         "action_required":    "Select the interpretation that
151                               applies, or rephrase.",
152     }

```

Listing A.6: `llm_agent.py` (`agent/llm_agent.py`), abridged. The sampling wrapper, the retrieval simulation, and the `ClarificationAgent`, whose structured output is the FM1-quality guarantee of Section 4.2.3. The `AnswerAgent`, `RetrievalAgent` and `HumanEscalationAgent` are thin deterministic wrappers of the same shape and are omitted here; all four are in the accompanying repository.

Retrieval simulation caveat. `simulate_tool_call()` does not perform live retrieval-augmented generation: for controlled evaluation it returns the dataset’s gold supporting context (HotpotQA) rather than querying a search index. The method signature and call pattern are identical to a production RAG call, so `RetrievalAgent.retrieve()` can be pointed at a real retriever without any change to the router; this substitution is flagged here rather than left implicit, since it affects how the epistemic-resolution results in the results chapter should be interpreted.

Agent design. Each specialist agent corresponds to exactly one `RoutingDecision` value (Listing A.4), so the router in Listing A.5 never needs conditional logic beyond a single dispatch on that enum. `ClarificationAgent` and `HumanEscalationAgent` share the same output schema (an `options/candidates` list of `{option, answer, support_count, support_fraction}` dicts) so downstream metrics code does not need to special-case which agent produced an escalation.

# Bibliography

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023.
- [2] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- [3] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, March 2023.
- [4] Jiaxin Zhang, Prafulla Kumar Choubey, Kung-Hsiang Huang, Caiming Xiong, and Chien-Sheng Wu. Agentic uncertainty quantification, 2026.
- [5] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023.
- [6] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation, 2023.
- [7] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 06 2024.
- [8] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms, 2024.
- [9] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know, 2022.
- [10] Armen Der Kiureghian and Ove Ditlevsen. Aleatory or epistemic? does it matter? *Structural Safety*, 31(2):105–112, 2009. Risk Acceptance and Risk Communication.
- [11] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision?, 2017.
- [12] Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Machine Learning*, 110(3):457–506, March 2021.
- [13] Yarin Gal, Riashat Islam, and Zoubin Ghahramani. Deep bayesian active learning with image data, 2017.

- [14] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR.
- [15] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017.
- [16] Stefan Depeweg, José Miguel Hernández-Lobato, Finale Doshi-Velez, and Steffen Udluft. Decomposition of uncertainty in bayesian deep learning for efficient and risk-sensitive learning, 2018.
- [17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017.
- [18] Shrey Desai and Greg Durrett. Calibration of pre-trained transformers, 2020.
- [19] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. How can we know when language models know? on the calibration of language models for question answering, 2021.
- [20] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
- [21] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models, 2023.
- [22] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words, 2022.
- [23] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback, 2023.
- [24] Yu Feng, Phu Mon Htut, Zheng Qi, Wei Xiao, Manuel Mager, Nikolaos Pappas, Kishaloy Halder, Yang Li, Yassine Benajiba, and Dan Roth. Rethinking llm uncertainty: A multi-agent approach to estimating black-box model uncertainty, 2025.
- [25] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11(53):1605–1641, 2010.
- [26] Christian Tomani, Kamalika Chaudhuri, Ivan Evtimov, Daniel Cremers, and Mark Ibrahim. Uncertainty-based abstention in llms improves safety and reduces hallucinations, 2024.
- [27] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation, 2023.
- [28] Alfonso Amayuelas, Kyle Wong, Liangming Pan, Wenhui Chen, and William Wang. Knowledge of knowledge: Exploring known-unknowns uncertainty with large language models, 2024.

- [29] Jeremy R. Cole, Michael J. Q. Zhang, Daniel Gillick, Julian Martin Eisenschlos, Bhuwan Dhingra, and Jacob Eisenstein. Selectively answering ambiguous questions, 2023.
- [30] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. A large annotated corpus for learning natural language inference, 2015.
- [31] Adina Williams, Nikita Nangia, and Samuel R. Bowman. A broad-coverage challenge corpus for sentence understanding through inference, 2018.
- [32] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021.
- [33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.
- [34] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.
- [35] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection, 2023.
- [36] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [37] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.
- [38] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet, 2024.
- [39] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024.
- [40] Jinwei Su, Qizhen Lan, Yinghui Xia, Lifan Sun, Weiyou Tian, Tianyu Shi, Xinyuan Song, Lewei He, and Yang Jingsong. Difficulty-aware agentic orchestration for query-specific multi-agent workflows, 2026.
- [41] David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer, 2018.
- [42] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert, 2021.

- [43] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering, 2018.
- [44] Sewon Min, Julian Michael, Hannaneh Hajishirzi, and Luke Zettlemoyer. Ambigqa: Answering ambiguous open-domain questions, 01 2020.
- [45] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Christopher Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 03 2019.
- [46] Ola Shorinwa, Zhiting Mei, Justin Lidard, Allen Z. Ren, and Anirudha Majumdar. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions, 2025.
- [47] Siddhartha Devic, Tejas Srinivasan, Jesse Thomason, Willie Neiswanger, and Vatsal Sharan. From calibration to collaboration: Llm uncertainty quantification should be more human-centered, 2025.
- [48] Michael J. Q. Zhang and Eunsol Choi. Situatedqa: Incorporating extra-linguistic contexts into qa, 2021.
- [49] Michael Kirchhof, Gjergji Kasneci, and Enkelejda Kasneci. Position: Uncertainty quantification needs reassessment for large-language model agents, 2025.
- [50] Dan Qiao, Binbin Chen, Fengyu Cai, Jianlong Chen, Wenhao Li, Fuxin Jiang, Zuzhi Chen, Hongyuan Zha, Tieying Zhang, and Baoxiang Wang. Epistemic gain, aleatoric cost: Uncertainty decomposition in multi-agent debate for math reasoning, 2026.
- [51] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks, 2019.
- [52] Jacob Cohen. A power primer. *Psychological Bulletin*, 112:155–159, 07 1992.
- [53] Sewon Min, Eric Wallace, Sameer Singh, Matt Gardner, Hannaneh Hajishirzi, and Luke Zettlemoyer. Compositional questions do not necessitate multi-hop reasoning, 2019.